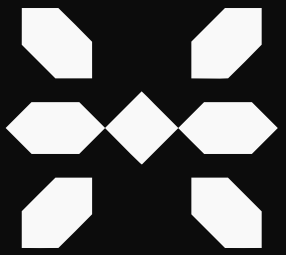
HOOX · AXIS · DOCUMENTATION



SYSTEMS ENGINEERS · CONTRIBUTORS

ARCHITECTURE MANUAL

ADRs, topologies, state namespaces, AXIS ≠ engine

Open charting PWA — own the axes, swap the engine

2026-11-01 DIN A4 · HTTPS://HOOX.SH/AXIS/DOCS OPEN SOURCE

CONTENTS

01 Overview

02 Overview

03 Evaluation

04 Adrs

05 Perf Audit 2026 11

06 Topologies

07 State Namespaces

08 Drawings and plot parity

09 Hp Engine

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Architecture" description: "AXIS system architecture: AXIS vs engine, plugin registry, topologies, ADRs, and state namespaces."

ARCHITECTURE

AXIS architecture is the discipline of **separable axes**—price history, live time, calculation, and library storage—coordinated by a Solid AXIS UI that never owns a closed interpreter.

Abstract

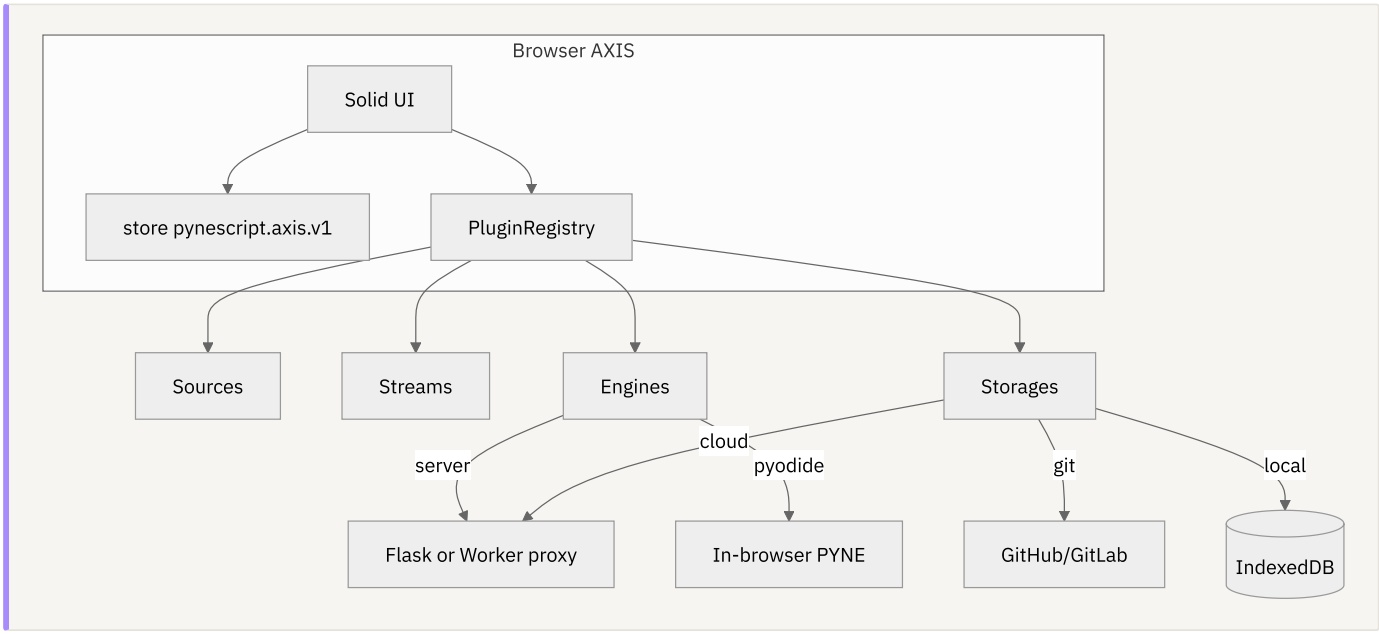
Layer	Responsibility
AXIS	UI, store, plugin orchestration, chart apply
Plugin contracts	`source | stream | engine | storage | dataset` (+ reserved component)
Engines	PYNE evaluation (browser Pyodide or remote `/run`)
On-chain plane	Parallel non-OHLCV datasets (TVL, DEX, events) via Worker allowlist proxy — ADR-015
Bar plane	Fine-grained OHLCV signals (`barPlane`) separate from `store.bars` — ADR-017
Optional edge	Cloudflare® Pages + Worker + DO/KV/D1/R2
Optional desk API	Flask Pro API

Invariant: AXIS ≠ engine. Evaluation always crosses an engine plugin boundary.

Track map

Page	Contents
Overview	End-to-end data/control flow
ADRs	ADR-001 … ADR-018
Perf audit	Q4 2026 performance & architecture plan
Topologies	Dev, static, edge deploy shapes
State namespaces	Keys, migration, hash, IDB
Drawings and plot parity	User tools vs Pine plots/drawings vs PYNE export
Live chart engine	Lightweight Charts paint, ingest worker, AXB1. The shared-memory ring and GPU overlay are gone

Conceptual model



Formal namespaces

```
Contract namespace:  pynescript.axis.plugins.v1
App state key:       pynescript.axis.v1
App state prefix:    pynescript.axis.*
Pages project id:    axis
Pages URL:           https://axis.hoox.sh
Worker script id:    worker-axis
Worker URL:          https://worker.axis.hoox.sh
Health service id:   worker-axis
```

Related tracks

- End User — operator workflows
- UI — UI subsystem design
- Plugins — contracts & catalogs
- Worker — edge data plane
- DevOps — build & CORS
- PYNE runtime — language evaluation

See also

- Root `README.md` — package-level map
- `LEGACY.md` — pre-Solid shell boundary
- AXIS CLI — operator install / deploy

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

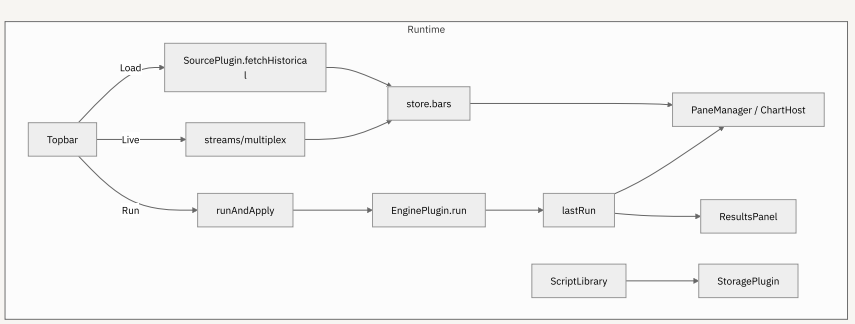
title: "Architecture overview" description: "End-to-end AXIS architecture: Solid AXIS UI, unified registry, three built-in engines, chart apply pipeline, and optional edge plane."

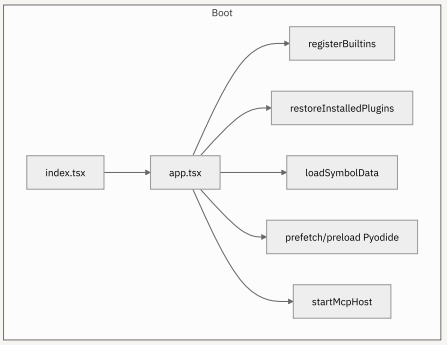
ARCHITECTURE OVERVIEW

Abstract

AXIS is a **composition host**. The product thesis: own the axes (history, live, calc, library), swap the implementations. Shipping UI is Solid + Vite + lightweight-charts + CodeMirror 6; calculation is never inlined into UI components.

Conceptual model





Interface surface (module map)

Concern	Primary paths
Entry	`src/index.tsx`, `app.tsx`
State	`src/store/`
Plugins	`src/plugins/{registry,types,loader,bootstrap,active}.ts`
Catalogs	`sources/catalog.ts`, `streams/*`, `engines/catalog.ts`, `storage/*`
Chart	`chart/ChartHost.tsx`, `pane-manager.ts`, `drawing-layer.ts`, `price-precision.ts`
Editor	`editor/*`
Run apply	`indicators/runner.ts`
Results	`results/*`, `ui/ResultsPanel.tsx`
Studio pages	`ui/studio/`, `ui/runtime/`, `ui/wire/`, `ui/settings/`, `ui/workers/`, `ui/plugins/`
Workers catalog	`workers/*`
Worker	`worker/`
CLI	`packages/cli/`
Desktop	`src-tauri/`, `src/desktop/`

Pre-Solid parallel (`state.js`, `registry.js`, `main.js`) was **removed in 2.21.0** — Solid + Vite is the only product path. See repo `LEGACY.md`.

Control plane vs data plane

Plane	What moves	Who owns it
Control	activePlugins, endpoint, theme, layout	Solid store + localStorage
Historical data	Bar arrays	Source plugins → `store.bars`
Live data	Bar updates	Stream plugins → multiplex → bars/chart
Calculation	script + bars → RunResult	Engine plugins
Library	ScriptDocument	Storage plugins
Telemetry	plane connectivity / latency / ticks	Ephemeral `store.telemetry` + Connection HUD

Transport preference (ADR-014)

Path	Preferred transport	Notes
Load (history)	REST / local	Venue kline HTTP, CSV, mock walk
Live	**WebSocket**	Venue WSS with reconnect; mock-poll local; DO = brokered
Engine server	HTTP	`POST /run` batch; latency in HUD
Engine pyodide	Local	Offline-capable after asset cache

Live pairs with history via `defaultStreamForSource`. `live.preferAfterLoad` auto-starts WS after Load and on boot when bars exist (default **on**).

Built-in engines

Engine	Transport	Fidelity host
`server`	WebSocket `/ws/run`, then `POST /run`	Flask Pro API or AXIS Worker
`pyne-worker`	`POST /run` (optional WS)	HOOX pyne-worker. Built-in. Not a pasted Backend URL on `server`.
`pyodide`	In-browser Python	Self-hosted Pyodide 0.29.5 + vendored wheel + `pynescript_runtime.py`

Timeout policy in runner scales with bar length (clamped). Live re-runs use shorter silent timeouts.

Chart apply pipeline

`runAndApply`:

1. `getActiveEngine().run(...)`
2. `setLastRun`
3. Optionally open results
4. **Sync** overlays in place (`syncOverlayLines`) — no blank destroy frame
5. Markers + strategy report side effects
6. Atomic Pine script drawings replace
7. Equity pane when appropriate (skip hide thrash on silent live re-runs)

History vs live data path: `loadBars` bumps `chartDataGen` → ChartHost full `setDataToChart` + fit. Live ticks only `manager.appendBar` (never full setData/fitContent).

PaneManager is imperative DOM under a Solid-owned outer shell—**Solid must not reconcile the pane root** (ChartHost invariant).

Plugin registry

`PluginRegistry` holds ordered maps per kind; events `registered` / `unregistered`. Built-ins register once (`ensureBuiltins` / catalog `ensure*Registered`). Dynamic plugins via `loadPluginFromUrl`. Active selection is **not** inside the registry—it is store state.

Contract namespace conceptually: `pynescript.axis.plugins.v1` (fields: `id`, `name`, `kind`, `configSchema`, `capabilities`, kind-specific methods).

Edge plane (optional)

Cloudflare® split:

- Pages project **`axis`** (`https://axis.hoox.sh`): static `dist/`
- Worker script **`worker-axis`** (`https://worker.axis.hoox.sh`): `/api/run` proxy, keys/usage KV, scripts D1, stream DO fan-out
- AXIS still speaks engine/storage plugins—not raw CF APIs in UI components

Invariants

1. AXIS ≠ engine
2. Bars/logs/lastRun not fully persisted
3. Storage and engine endpoints may coincide (Worker) but remain separate plugin roles
4. Older keys migrate forward, never the reverse on write

Failure modes (architectural)

Mode	Manifestation	Mitigation
God-object UI	Logic in components	Keep runner/registry pure modules
Engine leakage	Fetch `/run` from random widgets	Only engine plugins perform calc I/O
Registry/store split brain	Flat `engine` ≠ `activePlugins.engine`	`setActivePlugin` keeps alignment
SPA asset fallback	Pyodide loads HTML	Assert ZIP magic on assets

See also

- ADRs
- Topologies
- State namespaces
- AXIS store

EVALUATION

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Evaluation map" description: "How AXIS runs Pine: Flask Pro API, Worker proxy, in-browser Pyodide. AXIS does not import PyneTS and does not place HOOX orders."

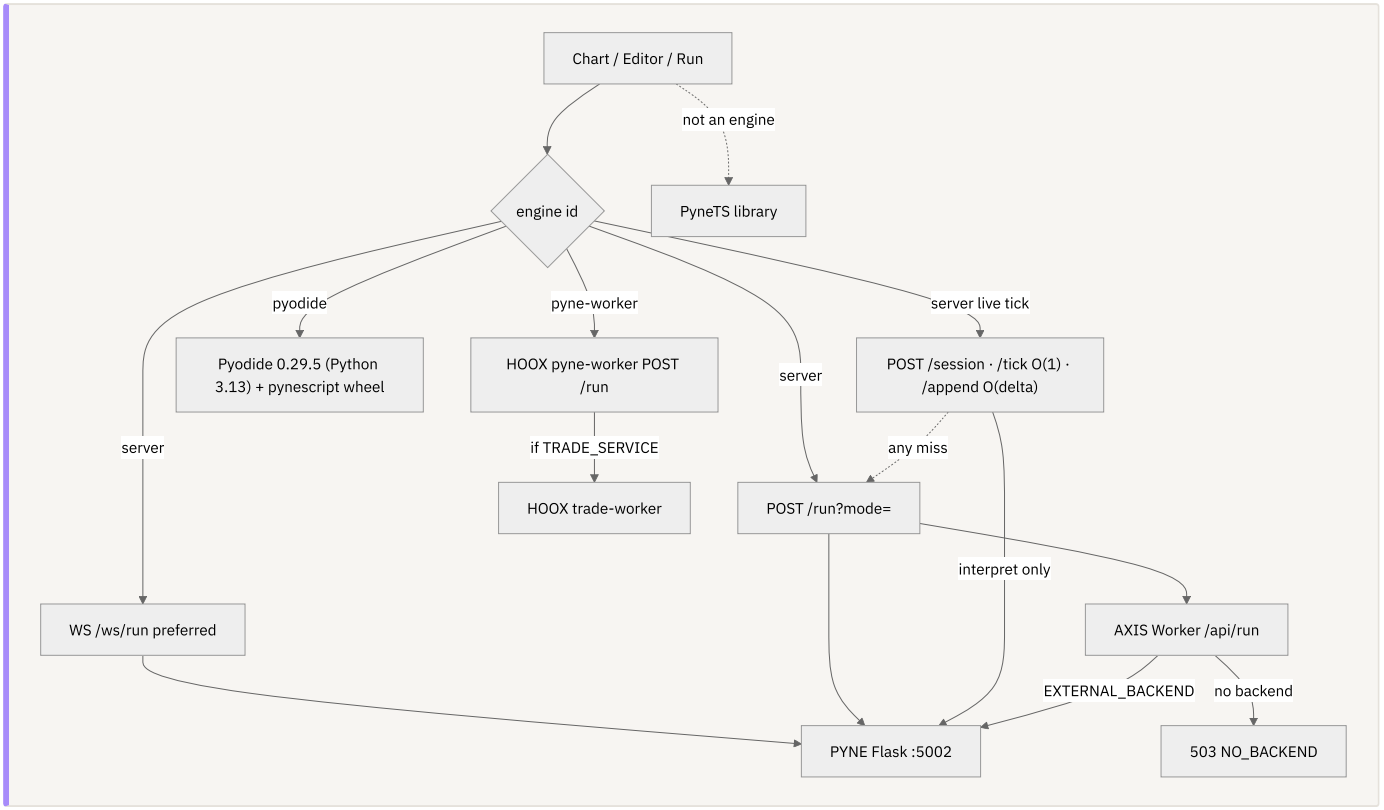
EVALUATION MAP

Abstract

AXIS is a charting PWA, not a language rewrite. Every Pine evaluation goes through an **engine plugin**. Built-ins are `server` (HTTP/WS to PYNE) and `pyodide` (in-browser Python wheel). Optional HOOX **pyne-worker** is just another Backend URL.

AXIS does not import `@hoox-sh/pynets`. AXIS does not call `trade-worker`. Drawing a strategy on the chart is research, not execution.

Conceptual model



Interface surface

Path	Implementation	Notes
Engine `server`	`src/engines/catalog.ts`	Prefer **WS** `ws(s)://{endpoint}/ws/run` (`preferWs` default true), else **POST** `{endpoint}/run?mode=` with `{ script, data, mode, inputs?, profiler?, libraries? }`. Modes `interpret | compile | auto`. Timeout scales with bar count (45–180s). Default endpoint **`https://pynescript.online`** (local desk override `http://127.0.0.1:5002`). Strategy HPO uses **`POST {endpoint}/optimize`**. **Live sessions** (Pro API 0.6.9+, interpret only): genuine live ticks (`EngineOpts.liveTick`) go through `src/engines/live-sessions.ts` — `POST /session`, `/tick` (O(1)), `/append` (O(delta)) — with silent fallback to the full run on any miss. Only engaged for `interpret` runs: the configured mode must be `interpret`, or the backend must have reported `interpret` for the last full run of the same script (`auto`). Scripts whose inputs hold expanded `input.source` series stay on `/run` — a session freezes its inputs at create.
Engine `pyodide`	same file	Self-hosted **Pyodide 0.29.5** (`/pyodide/v0.29.5/`, Python 3.13), wheels `/vendor/pynescript-0.6.29-py3-none-any.whl` + `antlr4_python3_runtime-4.13.2`, runtime `/pyodide/pynescript_runtime.py`. **Numba is not in Wasm**; `compile` / `auto` is object-mode. Cold load ~20–30s.
AXIS Worker `/api/run`	`worker/src/runtime.ts`	`PYODIDE_IN_WORKER=enabled` scaffold; else **proxy `EXTERNAL_BACKEND/run`**; else 503 `NO_BACKEND`. Caps: 512 KiB script, 50k bars, 30 req/min/IP, 60s upstream.
Engine `pyne-worker`	`src/engines/catalog.ts`	Built-in (`builtIn: true`). `POST /run` to the HOOX edge worker. Default origin `https://pyne-worker.cryptolinx.workers.dev`. Distinct from engine `server` and from the AXIS Worker (`worker-axis`).
PyneTS	—	TypeScript library. PYNE docs. **Not referenced** in AXIS source.

Payloads follow the PYNE evaluate contract. Defaults: Flask `POST /run` uses `mode` **`auto`**. See modes.

Internals

Path	Role
`src/engines/catalog.ts`	`server`, `pyne-worker`, `pyodide`
`src/engines/live-sessions.ts`	O(delta) live sessions (`/session`, `/tick`, `/append`); interpret-only, silent fallback to `/run`
`src/workers/catalog.ts`	Workers page catalog entries
`worker/src/runtime.ts`	Edge `/api/run`
`public/pyodide/`	Wasm runtime + wheel
`scripts/sync-pyne-wheel.sh`	Refresh vendored wheel

Invariants & edge cases

1. **AXIS ≠ engine** (ADR-002). The shell never embeds a closed interpreter.
2. **AXIS ≠ execution.** Strategy events on the Results panel are not HOOX orders. See AXIS and HOOX.

3. **No `none` stream plugin** in `src/streams/catalog.ts`. Pausing live data is `stopLive()`, not a plugin id.
4. The vendored wheel is `pynescript-0.6.29-py3-none-any.whl`, built from the sister pyne repo with `scripts/sync-pyne-wheel.sh`. That script also prints a warning if the hard-coded path in `src/engines/catalog.ts` drifts from the wheel filename — update it when the version changes.
5. CORS: Worker allows localhost, `*.hoox.sh`, `*.pynescript.online`, `*.axis.pages.dev` (plus legacy `*.pynescript-axis.pages.dev`) only.
6. **A live tick never changes evaluation semantics.** Live sessions (`src/engines/live-sessions.ts`) always evaluate in `interpret`, so the O(delta) path engages only for a run the backend itself resolved to `interpret` (the client records the mode each full run reports), and a session freezes its inputs at create — scripts whose inputs hold expanded `input.source` series stay on `POST /run`. Any miss falls back to the full run, and a response that is not a `/run`-shaped result is a miss, never an empty success.

Worked examples

Local desk

```
# terminal 1 — PYNE Pro API
cd ../pyne && make run    # :5002 — sister repo hoox-sh/pyne; some checkouts are still named pynesc

# terminal 2 — AXIS
cd axis && bun run dev
```

Manager → engine `server`, Backend URL `http://127.0.0.1:5002` (overrides the product default `https://pynescript.online`).

Offline

Switch engine to `pyodide`. First run downloads/caches the self-hosted runtime. Do not expect Numba.

Edge proxy

Deploy AXIS Worker with `EXTERNAL_BACKEND` pointing at Flask or pyne-worker. Engine stays `server`; endpoint is the Worker origin. Still no trade-worker hop unless **that** backend forwards events.

Pyodide in Worker (scaffold — not production yet)

Goal: a fully-CF deployment (no Flask) with the Worker evaluating Pine itself.

```
# worker/wrangler.toml
PYODIDE_IN_WORKER = "enabled"   # default "disabled" — leave it there for now
```

Status: Pyodide boots per isolate, but the wheel install + `run_script` bridge are stubbed, so every run returns an error envelope (HTTP 200) and does **not** fall through to `EXTERNAL_BACKEND`. Enabling the flag today breaks evaluation. Completion checklist (R2 wheel, `BUNDLES` binding, micropip install, bridge, fail-open semantics) lives in `worker/RUNTIME.md`. Production recommendation stays `EXTERNAL_BACKEND`.

Failure modes

Symptom	Cause	Fix
503 `NO_BACKEND`	Worker has no `EXTERNAL_BACKEND` and Pyodide-in-worker off	Set backend or use browser Pyodide
503 body `{ status: 'error', code: 'NO_BACKEND' }` to API consumers	Same as above, machine shape	Do **not** retry — the backend is absent, not busy. Fix config, then resend. (PYNE Agent treats this as skipped-with-note after 1 attempt.)
Compile "slow / different" in Pyodide	No Numba	Use Flask for numeric compile
Expecting pynets in the PWA	Not wired	Import `@hoox-sh/pynets` in your own app
Strategy PnL on chart, no CEX fill	AXIS does not execute	pyne-worker live trading

See also

- Engines
- Worker runtime
- PYNE Pro API
- PyneTS
- HOOX pyne-worker
- AXIS and HOOX

ADRS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Architecture Decision Records" description: "ADR-001 through ADR-018 for AXIS: registry, AXIS≠engine, Solid+Vite, engines, plugins, storage, CF ids, proxies, DO fan-out, CORS, results, telemetry, on-chain, provider lock, narrow bar plane, engine result parse."

ARCHITECTURE DECISION RECORDS

Formal ADRs for AXIS (product + the repo root). Status values: **Accepted** unless noted. Dates are conceptual product epochs, not git archaeology.

Index

ID	Title
ADR-001	Pluggable unified registry
ADR-002	AXIS ≠ engine
ADR-003	Solid + Vite product UI
ADR-004	Three built-in engines (server, pyne-worker, Pyodide)
ADR-005	Declarative configSchema
ADR-006	URL-loadable plugins
ADR-007	Storage as a plugin
ADR-008	CF ids: Pages `axis`, Worker `worker-axis`
ADR-009	Worker proxies first
ADR-010	Durable Object stream fan-out
ADR-011	State namespace migration
ADR-012	CORS as deploy concern
ADR-013	Results export in AXIS
ADR-014	Transport preference + connection telemetry
ADR-015	On-chain data plane
ADR-016	Provider-locked market data; CCXT on the server
ADR-017	Narrow bar plane (`barPlane`) separate from `store.bars`
ADR-018	Off-thread engine JSON parse + typed result contract

ADR-001: Pluggable unified registry

Context

Early AXIS registered sources/streams/engines in separate ad hoc maps. Operators and authors needed one mental model and one install path.

Decision

Adopt a **unified** **`PluginRegistry`** (`src/plugins/registry.ts`) with kinds:

`source | stream | engine | storage | component(reserved)`

Every plugin shares `PluginBase` (`id`, `name`, `kind`, `description`, `version`, `builtIn`, `configSchema`, `capabilities`, lifecycle hooks). Registration is ordered; listeners observe register/unregister.

Consequences

- Active selection lives in the Solid store, not the registry.
- Built-ins protected from casual unregister.

- Catalogs (`sources/catalog.ts` , ...) become registration facades.
- Contract docs under Plugins.

ADR-002: AXIS ≠ engine

Context

Closed chart hosts couple rendering and language runtime. That blocks offline mode, alternate evaluators, and headless reuse of PYNE.

Decision

AXIS never embeds a closed interpreter. All evaluation goes through `EnginePlugin.run` . UI modules call `getActiveEngine()` / `runAndApply` , not Python bindings directly (except inside the pyodide engine module).

Consequences

- Language fidelity owned by PYNE.
- AXIS docs do not duplicate grammar theory.
- New engines (tiny demo, remote Worker, future WASM) are additive plugins.

ADR-003: Solid + Vite product UI

Context

Pre-2.21.0 vanilla JS shell (`main.js` , `style.css`) mixed DOM wiring with business logic and aged TV-blue tokens. Removed in 2.21.0.

Decision

Product path is **SolidJS + Vite + Tailwind 4**, entry `src/index.tsx` / `app.tsx` . Imperative chart library (lightweight-charts) is isolated in `ChartHost` / `PaneManager` so Solid reconciliation does not thrash canvas DOM.

Consequences

- Legacy files retained for smoke/static tests only (`LEGACY.md`).
- Icons via `lucide-solid` .
- Deploy artifact is Vite `dist/` , not repo-root static tree.

ADR-004: Built-in engines

Context

Researchers need server fidelity, an edge evaluator, and offline demos. A single transport cannot satisfy all three.

Decision

Ship three built-in engines (`BUILTIN_ENGINES` in `src/engines/catalog.ts`). `pyne-worker` is `builtIn: true` . It is not a URL you paste onto the `server` engine.

Id	Role
`server`	WebSocket `/ws/run` , then `POST {endpoint}/run` . Default endpoint `https://pynescript.online` (override to local Flask `:5002` or the AXIS Worker).
`pyne-worker`	HOOX edge evaluator. Same `/run` contract, default origin the production pyne-worker.
`pyodide`	In-browser Python + vendored pynescript / antlr wheels.

All three implement `EnginePlugin` and return `RunResult` .

Consequences

- Endpoint field only for server-like engines.
- Pyodide requires correct static asset deployment (ZIP integrity checks).
- Timeouts unified at runner layer.

ADR-005: Declarative configSchema

Context

Per-plugin settings UIs do not scale; custom React/Solid forms per plugin forks the manager.

Decision

Plugins declare optional **`configSchema`** : map of field name → `{ type, default, label, options, min, max, ... }` . Settings and library panels resolve defaults + `pluginsConfig` overrides via shared resolve helpers.

Consequences

- New plugins get settings UI without shell changes (within field types).
- Types: `string | number | boolean | select` initially.
- Invalid user values remain a plugin responsibility at runtime.

ADR-006: URL-loadable plugins

Context

Third-party sources/engines should not require rebuild of the PWA for experiments.

Decision

Support **`loadPluginFromUrl`** : fetch ES module, validate shape, register, persist install metadata for restore on boot. Example plugins published under `public/plugins/` .

Consequences

- Browser origin trust model: URL plugins are code execution.
- Security tests constrain schemes and storage-via-URL footguns.
- Offline restore needs prior successful fetch/cache.

ADR-007: Storage as a plugin

Context

Script library began as localStorage-only; cloud and git were bolted differently.

Decision

Storage is a first-class plugin kind with `list/read/write/remove` (+ optional draft/sync/status). Built-ins: `local` (IDB), `cloud` (Worker `/api/scripts`), `git` (GitHub/GitLab Contents API). Active storage in `activePlugins.storage`.

Consequences

- Manager Script Library is backend-agnostic.
- Git commits only on explicit save; drafts local.
- Cloud uses Bearer Pro keys and optional optimistic concurrency.

ADR-008: Frozen CF project id

Context

Renaming Cloudflare® Pages/Worker projects severs KV/D1/R2 bindings, custom domains, and CI secrets. Product brand evolved to AXIS. The original freeze used a single id `pynescript-axis` for both Pages and the Worker.

Decision

Pages project is `axis`. Canonical URL `https://axis.hoox.sh` (custom domain). Preview hashes live on `*.axis.pages.dev`. The previous Pages id `pynescript-axis` remains a CORS legacy alias for old preview hosts.

Worker script is `worker-axis`. Canonical URL `https://worker.axis.hoox.sh` (custom domain). Health JSON `service` is `worker-axis`. The pre-rename workers.dev host (`pynescript-axis.<account>.workers.dev`) is treated as a legacy alias in client host matching.

Brand in UI/manifest/docs is AXIS.

Consequences

- `wrangler.toml` `name` is `worker-axis`; Pages `--project-name` is `axis`.
- Canonical hosts: Pages `https://axis.hoox.sh`, Worker `https://worker.axis.hoox.sh`.
- Docs call out the split explicitly (this ADR).
- npm package name (`axis-worker`) may lag brand.

ADR-009: Worker proxies first

Context

In-Worker Pyodide is attractive but heavy; production needed a path immediately.

Decision

Worker **`/api/run` proxies to an external Python backend** first (`EXTERNAL_BACKEND`). In-Worker runtime remains scaffolded/gated. Keys, usage metering, scripts API, and stream DO can ship independently of full edge evaluation.

Consequences

- AXIS `server` engine talks to Worker URL transparently.
- Operational dependency on Flask (or equivalent) until edge runtime matures.
- Clear layering: Worker is data plane + auth, not necessarily the interpreter.

ADR-010: Durable Object stream fan-out

Context

N browser clients each opening venue WebSockets waste connections and risk rate limits.

Decision

`SessionDO` Durable Object: one upstream WS per session key; fan-out to N client sockets via `/api/stream`. Hibernation-friendly design on CF.

Consequences

- Stream plugins may target DO URLs (example plugin provided).
- Session/symbol/interval query params are part of the contract.
- Not a historical source—live only.

ADR-011: State namespace migration

Context

Hard-renaming storage namespaces would brick user layouts and libraries if keys changed hard.

Decision

Canonical key **`pynescript.axis.v1`**. On read, fall back to `pynescript.axis.v2` (and parallel library/draft keys), then **write forward**. Do not keep dual-write forever.

Consequences

- Seamless upgrade for existing browsers.
- Persistence strips bars/lastRun/logs.
- Tests cover migration (`tests/state.test.ts` patterns).

ADR-012: CORS as deploy concern

Context

Browser AXIS on origin A calling Pro API on origin B fails without CORS. Hardcoding origins in AXIS is wrong.

Decision

CORS is a backend/deploy configuration (`ALLOWED_ORIGINS` on Flask; Worker headers as deployed). AXIS documents required origins; ops sets explicit lists in production (avoid perpetual `*` outside demos).

Consequences

- Localhost regex for dev.
- Troubleshooting centers on preflight, not engine code.
- Static file server does not replace API CORS.

ADR-013: Results export in AXIS

Context

Researchers need artifacts (trades CSV, run JSON) without a separate analytics service.

Decision

Results panel owns export: download JSON of `lastRun`, CSV of closed trades, clipboard helpers. Strategy pairing and stats run client-side from engine events (`results/strategy.ts` , `results/events.ts`).

Consequences

- Export fidelity bounded by engine event quality.
- No server-side report store required for MVP.
- Viewer metrics ≠ broker statements (documented for operators).

ADR-014: Transport preference + connection telemetry

Context

Operators need to see **how** data and calculation move (WS vs REST vs local vs brokered), whether the live socket is actually open, and engine run latency. Naively “prefer WebSocket everywhere” is wrong: venues expose **history over REST** and **live klines over WSS**. Server engines batch-run over HTTP; Pyodide is local WASM.

Decision

1. **Transport policy**
   - History → REST (or local mock/CSV).
   - Live → **WebSocket first**, paired per source via `defaultStreamForSource` (`binance-rest` → `binance-ws` , …).

- Engine → **prefer** `WS /ws/run` on the Pro API when available (`flask-sock`); fall back to HTTP `POST /run`. Pyodide remains local.
- Brokered → CF Durable Object / `needsProxy` streams surface as transport class `broker`.

2. **Honest stream state** — `connecting` until `onStatus({ state: 'open' })`; reconnect uses `reconnecting` / degraded telemetry, not false green.
3. **Venue streams use reconnect with exponential backoff** (`streams/reconnect-ws.ts`).
4. **Connection HUD** — StatusBar second row (`ConnectionHud`) reads ephemeral `store.telemetry` (SRC/STR/ENG/STO chips, tick pulse, engine latency). Persist only `telemetry.hud` prefs + `live.preferAfterLoad` / `live.rerunOn`.
5. **Live re-run modes** — `every-tick` (default) or `bar-close` (venue `Bar.closed` or bar time advance).
6. **Chart stability** — full `setDataToChart` only on history load (`chartDataGen`); live uses `appendBar`; overlay/script drawings update in place to avoid hide/show flash.

Consequences

- UI never claims WS for historical load.
- Auto-live after Load is **on by default** (`preferAfterLoad`). Turn off in Settings → Live stream.
- Plugin capabilities may declare `transport`; otherwise HUD classifies from id/capabilities.
- See Streams, UI shell, Overview.

ADR-015: On-chain data plane

Context

AXIS began as an **OHLCV-first** chart host: `SourcePlugin` / `StreamPlugin` deliver `Bar[]` (open, high, low, close, volume) for CEX and similar venues, then engines run Pine against that series. Operators also need **non-price market structure**:

Need	Why `Bar` alone fails
Protocol TVL (USD)	Single scalar liquidity total, not OHLC
DEX pool candles	Same shape as bars, but different venue semantics, address/network keys, and page caps
Spike / unlock-style events	Discrete time markers with severity, not candles
Honest provenance	Provider, snapshot cadence, finality — not implied by “the chart symbol”

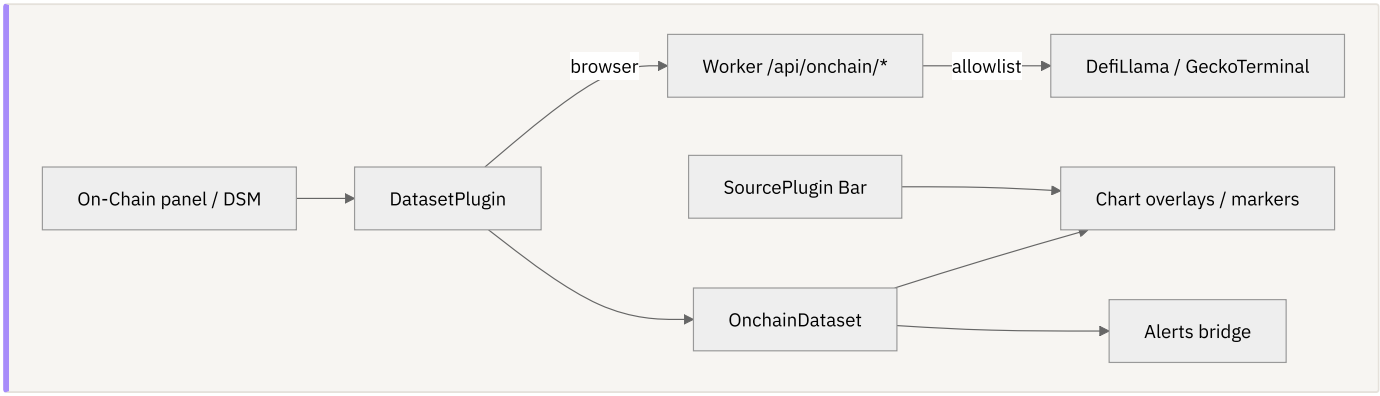
Forcing these into the primary `Bar` pipeline would corrupt scale semantics (TVL vs price), invent fake OHLC for scalar metrics, and couple wallet/signing UX to read-only analytics.

Decision

Ship a **parallel on-chain data plane** orthogonal to the OHLCV source/stream path:

1. **Plugin kind** `dataset` on the unified registry (`DatasetPlugin` in `src/plugins/types.ts`):
   - `fetchDataset(opts) → OnchainDataset`
   - optional `searchInstruments`
   - payload kinds: `ohlcv | scalar_series | multi_series | events | table`
   - required **provenance** + **finality** (`pending | safe | finalized | unknown`)

2. **Parallel plane modules** under `src/onchain/` (types, cache, keys, adapters, catalog, manager, events, alerts bridge) — not a new `Bar` field and not Pine output.
3. **Worker allowlist proxy** (`worker/src/onchain.ts`, routes under `/api/onchain/…`):
   - rewrite only known DefiLlama / GeckoTerminal paths
   - validate network ids and pool addresses
   - short-TTL isolate cache (`X-Axis-Onchain-Cache`)
   - client default bases from `store.endpoint` (`src/onchain/proxy.ts`)
4. **Chart UX**: attach as host-side overlays (left scale for scalars; separate series keys `onchain_*`); main market OHLCV stays on the right scale. Not wallet, not Pine plots.



Alternatives considered

Alternative	Rejected because
Force into `Bar` / active source	TVL is not OHLC; faking open=high=low=close lies to engines and the price scale. Multi-protocol attach becomes multi-symbol load thrash.
Piggyback Pine `request.security` only	Needs engine + script for every host overlay; no attach UI, no provider search, no finality UI.
Browser → provider direct	Public DefiLlama / GeckoTerminal hosts often lack CORS for arbitrary AXIS origins (ADR-012).
Open reverse proxy (any URL)	SSRF risk; Worker must **allowlist** path shape, not pass-through arbitrary upstream.
Wallet-connected RPC as the plane	Different product (signing, accounts, private RPC). On-chain here means **public analytics APIs**, not MetaMask.

Consequences

- **CORS / deploy** — browsers should use `{endpoint}/api/onchain/llama|gecko`; local Worker on `:8787` or deployed edge. Direct `baseUrl` overrides are for Node tests / trusted environments only.
- **Finality honesty** — DefiLlama TVL is typically a **daily snapshot**, not block-final chain state; Gecko OHLCV is **pool** sampling, not CEX marks. UI shows provenance lines; `synthetic` flags fabricated OHLC.
- **Not a wallet** — no signing, no injected providers, no private keys for this plane.
- **Not Pine** — overlays are host datasets; they do not replace `indicator()` / `strategy()` plots from the engine.
- **Engines unchanged** — active OHLCV source still feeds `EnginePlugin.run`; on-chain series do not silently become script `close`.
- **Cache** — client IDB dataset cache (`instrumentCacheKey`) plus Worker short-TTL memory; operators may see stale TVL briefly after large DeFi moves.
- **Security** — Worker rejects path traversal, invalid slugs/addresses, non-GET, and non-allowlisted Gecko paths.

Phases shipped

Phase	Capability	Primary surfaces
1 — TVL	DefiLlama protocol TVL scalar history + search	`defillama-tvl` dataset, `src/onchain/defillama.ts`, llama proxy
2 — DEX	GeckoTerminal pool OHLCV + pool search	source `geckoterminal-ohlcv`, `src/onchain/geckoterminal.ts`, gecko proxy. Not a `DatasetPlugin`.
3 — Events	Derived day-over-day TVL spike/drop events (`tvl_spike` / `tvl_drop`)	`src/onchain/events.ts` → chart markers
4 — Alerts	Bridge events into alerts engine (`onchain_tvl_spike` / `onchain_event`)	`src/onchain/alerts-bridge.ts`, `src/alerts/engine.ts`

Pro-only DefiLlama surfaces (raises, unlocks via `pro-api.llama.fi`) remain out of the free Worker allowlist.

Code map

Path	Role
`src/onchain/types.ts`	`OnchainDataset`, instruments, finality
`src/onchain/catalog.ts`	Built-in dataset registration
`src/onchain/manager.ts`	Attach / hide / detach store wiring
`src/onchain/proxy.ts`	Resolve Worker llama/gecko bases
`worker/src/onchain.ts`	Allowlisted proxy + health
`src/plugins/types.ts`	`DatasetPlugin` contract

Related docs

- Datasets — plugin contract
- On-Chain data (end user) — operator guide
- Worker — `/api/onchain/…` on the edge route table
- ADR-001, ADR-009, ADR-012

ADR-016: Provider-locked market data

Context

AXIS composes source × stream × engine, but **OHLCV aggregation is not composable**. Mixing Binance history with OKX live (or Kraken watchlist quotes on a Binance chart) silently poisons indicators, compare, watchlist, and cache. Prior to this ADR, source and stream were independently selectable, venue detection used plugin-id substrings, and the Binance synthetic fallback made network errors look like real data.

Key problems:

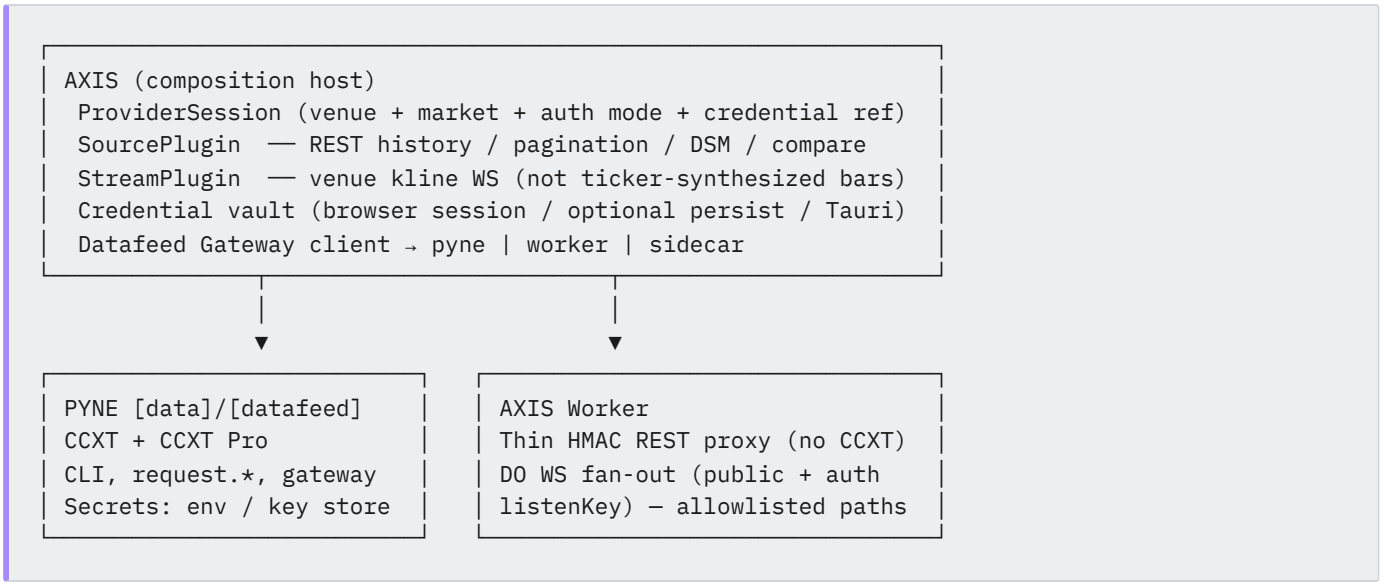
Issue	Risk
Source and stream independently selectable in Topbar	User loads Binance REST + OKX WS — mixed candles
DSM `sourceId` seeded once from `store.source`	Backfill targets a different venue than the chart
Watchlist Kraken → Binance tickers	Mixed-provider quotes
Coinbase WS ticker → synthetic bars	Not exchange candles; OHLCV ≠ REST history
Binance `fallback: true` synthesizes random walk	Chart looks "live" with fake prices

Decision

Introduce **`ProviderSession`** — a first-class locked market-data identity that all aggregators inherit:

1. **`ProviderSession`** (`src/data/provider.ts`): `{ id, sourceId, streamId, venue, market, authMode, credentialId, gateway }`. Persisted without secrets. Every aggregator reads `getActiveProvider()` instead of raw URLs or plugin-id substrings.
2. **Auto-pairing**: Source changes always re-pair the stream via `defaultStreamForSource`. Explicit mismatch is allowed but shows a HUD `⚠ pair · Fix` chip.
3. **Plugin capabilities**: `venue`, `market`, `klineStream` on `PluginCapabilities`. Venues with `klineStream: true` emit exchange candles (not ticker buckets).
4. **Coinbase WS**: switched from ticker buckets to Advanced Trade venue candles, folded into chart interval via `foldVenueCandle`.
5. **Binance fallback**: defaults **off** (`fallback: false`). Network errors must not look like real data.
6. **Watchlist lock**: no silent Binance fallback for Kraken, Gecko, or unknown venues. Each shows a clear "no live mux" message.
7. **Credential vault** (`src/data/credentials.ts`): in-memory session-only vault for API key/secret/passphrase. Secrets never reach `persist()`, `pluginsConfig`, error-share, or CHANGELOG dumps. `redactSecrets` and `stripSecretKeys` enforce this.
8. **Venue HMAC signers** (`src/data/venues/`): thin Binance / OKX / Bybit / Coinbase / Kraken / MEXC signers. No CCXT in the browser.
9. **Signed Binance klines**: vault key → HMAC direct fetch → Worker `/api/market/binance/signed/klines` (request-scoped `X-Exchange-Key` / `X-Exchange-Secret`). 401/403 stays thrown — bad keys must not silently mix public klines.
10. **Cache key**: includes `providerId|market|authMode|symbol|interval`. Authenticated and public candles do not share cache slots.

Ownership



Alternatives considered

Alternative	Rejected because
Bundle CCXT in browser	~1 MB+ bundle, Node APIs, isolate CPU, CORS issues
CCXT in Cloudflare® Worker	Bundle size, 128 MB memory, shared rate-limit IP, not designed for 100-exchange routing
Keep venues as raw URL substrings	Silent venue mismatches poison aggregation correctness
Pluggable venue resolver	Over-engineered; first-party venues are a fixed set of 5

Consequences

- **Correctness** — aggregation (indicators, compare, watchlist, cache, DSM) inherits a locked venue. Mixed-provider bugs are structurally prevented.
- **Auth path** — public remains the default. Auth is a capability upgrade on the same provider, not a second source id.
- **Secret safety** — vault is RAM-only in v1. Tauri keychain / AES-GCM durable wrap is Phase 1.5. `pluginsConfig` and error-share are stripped of secret keys.
- **CCXT stays in PYNE** — server-side only. AXIS calls the gateway with session handles, never raw secrets in `/run`.
- **Long-tail exchanges** — AXIS `ccxt-rest` / `ccxt-ws` plugins that talk to a PYNE or Bun sidecar gateway. First-party venues stay thin native adapters.

Key files

Path	Role
`src/data/provider.ts`	`ProviderSession` type, `buildProviderSession`, `defaultStreamForSource`
`src/data/credentials.ts`	In-memory vault, `putCredential`, `redactSecrets`
`src/data/signed-fetch.ts`	Vault → HMAC → direct or Worker proxy
`src/data/venues/`	Per-venue HMAC signers (binance, okx, bybit, coinbase, kraken)
`src/plugins/active.ts`	`getActiveProvider()`
`src/plugins/types.ts`	`PluginCapabilities.venue`, `.market`, `.klineStream`
`src/store/index.ts`	`setActivePlugin` auto-pairing, `syncProviderSession`
`src/ui/ConnectionHud.tsx`	Venue label, pairing Fix button
`src/ui/SettingsDialog.tsx`	Data tab (exchange credentials)
`src/ui/error-share.ts`	`stripSecretKeys` redaction
`worker/src/market.ts`	Signed Binance klines proxy

Related docs

- Sources — source plugin catalog
- Streams — stream plugin catalog
- Data Source Manager — DSM guide
- Topologies — composition models
- ADR-001, ADR-009

ADR-017: Narrow bar plane

Context

`AppState.bars` is a single Solid store array. A live tick that replaces or mutates `store.bars` notifies every `createEffect` / `createMemo` that read `store.bars` or `store.bars.length` — document title, status bar, quote card, volume profile, ChartHost overlays, even when they only need the forming tip. Phase 1 of the Q4 2026 perf audit measured that as the dominant reactivity cost on the live path.

Keeping one giant array as both the engine input and the UI subscription surface cannot stay under an 8 ms tick→paint budget at 5k–50k bars.

Decision

Introduce a **narrow bar plane** at `src/stores/bars.ts` (`barPlane`) with four independent fields:

Field	Updates when	Typical readers
`bars`	full load, slot restore, rolling-window slide	engine apply, full-window paint
`tip`	every accepted live tick (cloned, never the same object as `bars[n-1]`)	last price, title, quote, HUD
`length`	grow or slide, not forming-tick overwrite	chrome that only needs count
`gen`	full reload / slide	result cache key, overlay invalidation

Write path:

- `loadBars` / slot restore → `setBarPlaneFull`
- in-place append → `setBarPlaneAppend` (no whole-window copy)
- forming tick → `barPlane.tip` only (`flushLiveFrame`)

`store.bars` remains the durable engine/eval source during migration. New UI readers subscribe to `barPlane`, not `store.bars`.

The tip is **cloned** on full-window writes so the Solid store proxy for `barPlane.tip` cannot alias an element of `store.bars` (that aliasing broke `closed` mutation in `store-closed-on-new-bar.test.ts`).

Alternatives considered

Alternative	Rejected because
Fine-grained signals on `store.bars[i]`	Solid store array replace still notifies length/identity readers; does not split tip from window.
Drop `store.bars` immediately	Engine eval, DSM, and ChartHost paint still consume the full window; a hard cut would fork persistence.
Shared typed-array buffer	Follow-up (Phase 2 GC hygiene). Plane first, buffers later.

Consequences

- Live ticks that only move the forming OHLC must not bump `barPlane.gen` or copy `barPlane.bars`.
- Overlay/result caches key off `barPlane.gen` + `barPlane.length`, not array identity.
- Persistence still serializes `store.bars`. Do not persist the plane separately.
- ChartHost's full paint path may keep reading `store.bars` until a later phase moves it.

Key files

Path	Role
`src/stores/bars.ts`	`barPlane`, `setBarPlaneFull`, `setBarPlaneAppend`
`src/store/index.ts`	`loadBars` / `appendBar` / `flushLiveFrame` writers
`src/app.tsx`, `src/ui/StatusBar.tsx`, `src/ui/Topbar.tsx`	migrated readers
`src/indicators/run-cache.ts`	cache key includes `barsGen` + length

Related docs

- Performance audit — Phase 1 notes

- State namespaces
- ADR-011

ADR-018: Engine result parse + typed contract

Context

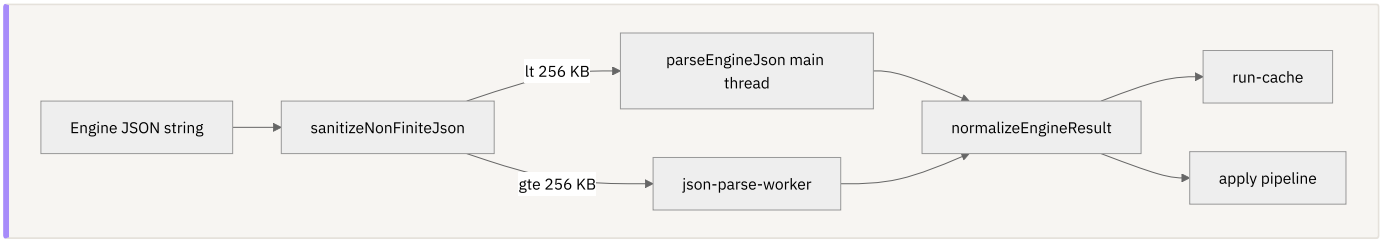
Engine responses (`POST /run`, `/ws/run`, Pyodide `run`) return one JSON string that can be hundreds of KB to several MB (equity, plot series, trades). `JSON.parse` of a 10k-bar payload was measured at p99 ~7 ms on the main thread — most of the 8 ms tick→apply budget — before apply even starts. Python `json.dumps` can also emit bare `NaN` / `Infinity`, which standard JSON rejects.

A second cost: REST, WebSocket, and Pyodide used to present slightly different result shapes to the apply pipeline, so each path re-normalized plots, trades, and equity.

True *chunked* streaming of Pyodide evaluation is still outstanding (Phase 5 backlog). This ADR covers the parse/contract slice that already shipped.

Decision

1. **Shared sanitizing parser** (`src/engines/json-sanitize.ts`):
   - `sanitizeNonFiniteJson` rewrites bare `NaN` / `Infinity` / `-Infinity` to `null` without touching string literals.
   - `parseEngineJson` is the synchronous path (small bodies, 413 peeks, tests).
   - `parseEngineJsonAsync` parses payloads **≥ 256 KB** in a module Worker (`src/engines/json-parse-worker.ts`). Below the threshold, or when `Worker` is unavailable, it falls back to `parseEngineJson`.
2. **Typed apply contract**: every engine path lifts through `normalizeEngineResult` (`src/indicators/run-helpers.ts`) to a `NormalizedRunResult` before ChartHost/apply. Pyodide, REST `/run`, WS `/ws/run`, and `/run/batch` share that shape.
3. **In-memory result cache** (`src/indicators/run-cache.ts`): successful non-live runs keyed by `(engineId, scriptId, scriptHash, inputs, config, barsGen, barsLength, libraries)`. Bounded 16 entries / 30 s TTL; cloned on store and retrieve; disabled in tests and for live ticks.
4. **Not this ADR**: incremental Pyodide line/mark streaming (`TransformStream`). Parse-off-thread of the *final* JSON string is the current win.



Alternatives considered

Alternative	Rejected because
Always parse on main thread	10k OHLCV p99 ~7 ms already saturates the tick budget.
Always off-thread	Worker postMessage of a 1 KB status frame is slower than `JSON.parse`. Threshold 256 KB.
Streaming JSON parser on main	Extra dependency; Worker reuse is enough for the final blob.
SharedArrayBuffer / typed-array wire	Needs backend envelope changes; gated for a later contract rev.

Consequences

- Callers of `parseEngineJsonAsync` must `await`. Sync peeks stay on `parseEngineJson`.
- Worker startup is amortized via a shared module worker; tests call `_resetParseEngineJsonForTests`.
- Cache must not serve live ticks or mutated apply payloads (clone on write/read).
- Chunked Pyodide evaluation remains future work; do not advertise partial plots as complete.

Key files

Path	Role
`src/engines/json-sanitize.ts`	`parseEngineJson`, `parseEngineJsonAsync`, 256 KB threshold
`src/engines/json-parse-worker.ts`	Off-thread `JSON.parse` after sanitize
`src/indicators/run-helpers.ts`	`normalizeEngineResult` / `NormalizedRunResult`
`src/indicators/run-cache.ts`	Bounded memo of non-live runs
`src/engines/catalog.ts`, `live-sessions.ts`, `engine-ws.ts`, `batch.ts`	Call sites

Related docs

- Performance audit — Phase 3 notes
- Evaluation
- ADR-002, ADR-004

ADR lifecycle

New ADRs should:

1. State context, decision, consequences.
2. Cross-link code paths.
3. Avoid re-litigating ADR-002 without supersession note.

See also

- Overview
- Topologies
- Plugins contracts
- Datasets

PERF AUDIT 2026 11

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Performance & architecture audit — Q4 2026" description: "Prioritized gap analysis for turning AXIS into a production-grade, super-snappy, modern charting application."

PERFORMANCE & ARCHITECTURE AUDIT — Q4 2026

1. Baseline health

The repo is already in a strong performance culture:

- Type check passes (`bun run typecheck` clean).
- Unit + worker test suite passes: **4708 pass / 10 skip / 0 fail**.
- `lint` finishes with zero errors (warnings only).
- A dedicated firehose bench exists under `AXIS_BENCH=1` (`tests/bench/`).
- The v1 live chart path has explicit frame-budget guardrails (`e2e/v1-chart.spec.ts`, 50 ms `panFrame` ceiling).

- `CHANGELOG.md` shows recent O(n) fixes (pre-eval call scanning, bar append path no longer re-sanitizes clean bars, windowed merges are linear).

The objective now is to move from "already good" to "absolutely world-class": sub-16 ms feedback, 60 FPS during live streaming + pan/zoom, and a UI/UX that feels like a premium commercial product.

2. Architecture strengths to preserve

1. **Plugin boundary is respected.** `AXIS ≠ engine` is enforced through `src/plugins/*`, `src/engines/*`, `src/workers/*`. No Solid component imports PYNE directly.
2. **Live/paint split is explicit.** `src/chart/live-append.ts` owns the only chart paint path from a live tick, with coalesced flushes and ranging-aware `setData` suppression.
3. **Data Source Manager is backgrounded.** `src/data/data-source-manager.ts` does all network paging on detached async jobs; `PAGE_YIELD_MS` keeps a page from starving the event loop.
4. **Per-slot runtime is separated from global state.** `src/chart/chart-registry.ts` keeps per-slot bars/gen in a dedicated store while managers/layers are imperative refs.
5. **Lazy UI loading.** `src/app.tsx` splits heavy shells (`StudioHost`, `EditorPane`, `CommandPalette`, panels) with `lazy()`.

3. Top bottlenecks & weaknesses

3.1 Reactivity surface too coarse

- **Evidence:** `src/store/index.ts` is 1,300+ lines and exports many mutation helpers. `loadBars` (`src/store/index.ts:3252`) uses `batch()` but still replaces `store.bars` with a fresh array, which notifies any component/effect that reads `store.bars`.
- **Impact:** `App.tsx` updates `document.title` on every bar update (`bars` effect, `src/app.tsx:179`), and `ChartHost.tsx` has ~15 `createEffect`s that read global store paths. A live tick causes a chain of effect runs even when nothing visible changes.
- **Severity:** High.

3.2 ChartHost is an effect-dense component

- **Evidence:** `src/chart/ChartHost.tsx` contains 11+ `createEffect`s with `untrack()` fences. Several effects read `store.chartDataGen` / `store.bars.length` per live append to guard overlays/drawings/pins. Each effect re-evaluates independently.
- **Impact:** More effect scheduling means more microtasks, especially during streaming pan/zoom.
- **Severity:** High.

3.3 Overlay updates are unconditional

- **Evidence:** Compare overlay effect (`src/chart/ChartHost.tsx:529`) calls `applyCompareOverlay()` whenever tracked compare state changes, even if the visible OHLC suffix is identical. On-chain overlay effect (`src/chart/ChartHost.tsx:573`) maps a full reactive array of points on every `chartDataGen` bump.
- **Impact:** Redundant full-series writes to LWC, GC pressure, possible frame drops.
- **Severity:** Medium-High.

3.4 Live tick still propagates to non-visual store paths

- **Evidence:** `appendBar` (`src/store/index.ts:3564`) sets `store.live.needsRerun` and `store.live.lastBarTime` on every accepted tick. These paths are read by the live re-run scheduler and, transitively, by indicator state.
- **Impact:** Thrashing indicator state / re-run bookkeeping on every tick while eval is still running, causing unnecessary work.
- **Severity:** Medium.

3.5 Engine response parsing is main-thread synchronous

- **Evidence:** `src/engines/pro.ts`, `src/workers/pyodide-client.ts`, and `src/engine/*` pass JSON back to the main thread and parse with `JSON.parse`. Large strategy/equity arrays block the main thread.
- **Impact:** A large backtest or deep indicator run can freeze the chart for 50–300 ms while the result is deserialized and applied.
- **Severity:** High.

3.6 Memory pressure from long-lived transient objects

- **Evidence:** `validateJobBars` (`src/data/data-source-manager.ts:195-229`) still calls `repairBars` on the dataset when sorting is suspected. Array slices in `applyCompareOverlay`, `applyOnchainOverlays` and `series-factory.ts` allocate new objects on every effect run.
- **Impact:** Extra GC on long sessions / 50k+ datasets.
- **Severity:** Medium.

3.7 UI theming and layout effects are not batched

- **Evidence:** `src/ui/panels/DockColumn.tsx` reads `store.panelChrome` in three separate memos (`ids`, `width`, `bottomHeight`) plus `getHoverSlideExpandedMap()`. `src/chart/ChartHost.tsx:448` schedules a reflow effect on `store.panelChrome` and `store.chartLayout.mode`.
- **Impact:** A single panel resize may trigger multiple effect passes and reflow RAFs.
- **Severity:** Medium.

3.8 Accessibility & polish gaps

- **Evidence:** Micro-interactions rely on ad-hoc inline Tailwind classes (e.g., `src/ui/shortcuts/Feedback.tsx` uses a hard-coded 1200 ms timeout and no `prefers-reduced-motion`). Many modal/panel components do not expose `aria-labelledby` routes. Focus traps are implemented per-component rather than through a single focus manager.
- **Impact:** Power users notice inconsistent feedback; keyboard/screen-reader users hit friction.
- **Severity:** Medium.

4. Prioritized improvement plan

Phase 0 — Measure first (week 1)

1. Add runtime telemetry hooks:
   - `performance.mark` around `appendBar`, `setDataToChart`, engine request/response, and JSON parse.
   - Track effect-count per tick using a dev-only ProbeStore wrapper.

- Record `panFrame` / `rAF` budgets from the existing e2e harness.

2. Create a profiling issue template and baseline dashboard entry.
3. Establish decision rule: **no Phase 1 change ships without a before/after measurement**.

Phase 1 — Reactive architecture (weeks 2–3)

1. **Decouple bars from the giant app store.**
   - Move the canonical OHLCV series and live-tip metadata out of `store.bars` into a dedicated, narrow signal/store (`src/stores/bars.ts`).
   - Keep `AppState` as UI orchestration state only; derive any UI reads from the narrow store.
2. **Audit every `store.bars` reader.**
   - Replace `createEffect(() => { const n = store.bars.length; ... })` with fine-grained derived signals (`tip`, `lastPrice`, `dataGen`).
   - Throttle `document.title` updates to 250 ms or only on tip closure.
3. **Coalesce ChartHost effects.**
   - Merge overlay and drawing effects where possible; use a single `createEffect` with a discriminated notification object.
   - Guard every chart write behind a deep-equality check for the visible data window.
4. **Batch store mutations.**
   - Wrap every public mutation helper in `batch()` where not already done.
   - Eliminate dual writes to legacy flat flags + `panelChrome` in the same turn.

Phase 2 — Rendering & data-plane performance (weeks 3–4)

1. **Incremental series updates.**
   - Make `applyCompareOverlay`, `applyOnchainOverlays`, and indicator apply paths compute a diff and only call `series.update` / `setMarkers` for changed data.
   - Use typed arrays or shared flat buffers for large numeric series when calling into LWC.
2. **Ranging-aware overlay writes.**
   - Suppress overlay/marker full-rewrites while `isRanging()` is true, mirroring the candle slide logic.
3. **Lazy bar payloads.**
   - Reuse arrays between store and slot cache by slicing from the same immutable ring buffer; avoid `unwrap()` copies on each append.
4. **GC hygiene.**
   - Pool transient arrays in heavy paths (indicator apply preview, on-chain overlays) with size caps.

Phase 3 — Engine boundary improvements (weeks 4–5)

1. **Main-thread deser offloading.**
   - For large responses, parse engine JSON in a Worker (`src/workers/json-parse.worker.ts`) or use streaming `JSON.parse` (`async` chunked parser) for result arrays > 100 KB.
2. **Typed-result contracts.**

- Add engine result envelopes that separate metadata from numeric series, enabling transfer of typed arrays and reducing string overhead.

3. **Pyodide streaming.**
   - Evaluate whether `TransformStream` can push back partial line/mark results as they are produced, instead of one huge JSON string.
4. **Result caching.**
   - Cache indicator evaluation for unchanged `(scriptId, barsGen, inputs)`; invalidate only when one of those changes.

Phase 4 — UX / visual polish (weeks 5–6)

1. **Design system hardening.**
   - Ensure every color/spacing/radius uses the void Tailwind theme tokens in `src/index.css` (@theme); delete leftover hard-coded color literals.
   - Add `prefers-reduced-motion` support globally; remove layout-property CSS transitions and rely on `transform/opacity` only.
2. **Focus & keyboard system.**
   - Introduce a central focus manager for modals/panels/drawers (`src/ui/focus-trap.ts`) and use it from `Modal.tsx`, `StudioHost`, `AlertsPanel`.
   - Add visible focus rings that match the design tokens; audit focus order in the chart/workspace.
3. **Loading and feedback states.**
   - Unify skeletons/spinners under one component hierarchy.
   - Improve shortcut feedback to respect reduced motion and debounce rapid chords.
4. **Mobile / responsive refinements.**
   - Verify bottom-sheet panel portals honor safe areas; reduce touch-target fragility on the toolbar.

Phase 5 — Continuous quality gates (week 6+)

1. Add a `bun run perf:audit` script that runs unit/e2e + records selected benchmark numbers.
2. Add a CI gate that fails if `AXIS_BENCH=1` firehose exceeds a budget.
3. Document ADR for the new narrow bars store and result-streaming contracts.
4. Keep `CHANGELOG.md` updated per checkpoint rule.

5. Validation metrics

Metric	Current target	World-class target
`appendBar` → paint latency (p99)	< 8 ms per tick	< 2 ms
Live pan/zoom frame budget (e2e)	< 50 ms	< 16 ms
Engine result JSON parse blocking	< 100 ms	parse off main thread or < 8 ms
Reactors triggered per live tick	untracked	≤ 3
Lighthouse performance	not measured	> 90
Unit test pass rate	100%	100% with perf gates

6. Risks & mitigations

Risk	Mitigation
Splitting the store breaks persistence shape	Keep `store.bars` as a derived view for migration; persist canonical data through the narrow store only if needed
Off-thread JSON parsing increases latency for small results	Only offload when payload > threshold (50–100 KB)
Typed-array result contracts require backend changes	Gate behind feature flags; server-only first, Worker later
Too many UI polish changes regress accessibility	Add a11y lint / Playwright a11y spec

7. Related files

- Store: `src/store/index.ts`, `src/chart/chart-registry.ts`
- Rendering: `src/chart/ChartHost.tsx`, `src/chart/live-append.ts`, `src/chart/series-factory.ts`, `src/chart/manager-access.ts`
- Data plane: `src/data/data-source-manager.ts`, `src/data/merge-datasets.ts`, `src/data/dataset-validate.ts`
- Engine boundary: `src/engines/pro.ts`, `src/engine/router.ts`, `src/workers/pyodide-client.ts`, `src/workers/pyodide-protocol.ts`
- UI shell: `src/app.tsx`, `src/ui/panels/DockColumn.tsx`, `src/ui/shortcuts/Feedback.tsx`, `src/index.css`
- Editor: `src/editor/PyneEditor.tsx`
- Tests/bench: `tests/bench/`, `e2e/v1-chart.spec.ts`

8. Phase 0 / Phase 1 implementation notes

> Updated during the first pass — these items have landed on the `development` branch.

Phase 0 — Instrumentation (landed)

- Added a tiny telemetry module at `src/perf/telemetry.ts` that wraps `performance.mark` / `measure` and only activates in dev/test builds.
- Placed marks/measures on the current dominant costs so future work has a before/after signal:
  - `src/store/index.ts`: `perfMark('axis:loadBars:setStore')`, `perfMark('axis:appendBar:store')`.
  - `src/chart/manager-access.ts`: `perfMeasureSync('axis:setDataToChart', …)` around the full paint path.
  - `src/engines/json-sanitize.ts`: `perfMeasureSync('axis:parseEngineJson', …)` around the shared engine response parser.
- Deviations from the plan: the dev-only ProbeStore wrapper and effect-count probe were deferred. The existing `chart.lag` / `chart.bars` MCP app capabilities already expose live latency data, so we are using those plus the new marks for initial profiling.

Phase 1 — Narrow bar plane (landed)

- Created `src/stores/bars.ts` with four fine-grained accessors:
  - `bars` — full visible window (readers can still subscribe to the whole array).
  - `tip` — the forming/newest bar; updates independently of `bars` on every live tick.
  - `length` — window size; only changes on grow/slide, not on every tick.
  - `gen` — full-reload generation; bumped by `loadBars`/slot restores and rolling-window slides.
- Wired the plane from the existing authoritative paths:
  - `loadBars` pushes the validated view into `barPlane` (with symbol/interval/exchange).
  - `appendBar` pushes appends and slides into `barPlane` (`bumpGen: true` only for slides).
  - `restoreSlotBarsToActivePlane` restores slot cache via `barPlane`.
  - `flushLiveFrame` updates only `barPlane.tip` when a forming tick lands.
- Discovered and fixed a Solid-store proxy-link issue during the migration: when the same raw bar object was placed both in `barPlane.bars` and in `barPlane.tip`, *and* that object was also an element of `store.bars`, the two stores' proxies could interfere with the app store's `closed` mutation during index appends. The fix is to clone the plane's `tip` in `setBarPlaneFull` so the narrow plane never exposes the exact same object instance as both the tail of `bars` and the live tip.
- Migrated the highest-frequency UI readers from `store.bars` / `store.bars.length` to `barPlane`:
  - Shell chrome: `App.tsx` (document title, rAF-throttled), `StatusBar.tsx`, `Topbar.tsx`.
  - Float/overlay panels: `PricePanel.tsx`, `DataViewPanel.tsx`, `QuotePanel.tsx`, `VolumeProfileOverlay.tsx`, `TimePanel.tsx`.
  - Extras/modals: `CurrentPriceCard.tsx`, `HpoPanel.tsx`, `BarReplayControls.tsx`.
- `store.bars` remains the canonical source for engine evaluation, legacy components, and `ChartHost`'s full paint path during this migration. Phase 2 will decide whether to move the paint path to the plane or to the slot cache.

Tooling updates

- `package.json` now exposes `bun check` as the canonical typecheck command (root, worker, and CLI). The old `bunx tsc --noEmit` is retired for the standard alias.
- Fixed `packages/cli/tsconfig.json` where `baseUrl` now emits a TS5102 error; replaced it with `"paths": { "*": ["./*"] }` so `typecheck:all` passes cleanly.

Validation after Phase 0/1

- `bun run typecheck:all` — clean (563 app + 29 worker + 39 CLI files).
- `bun run lint` — exits 0 (warnings remain; no errors).
- `bun run test:unit` — **4708 pass / 10 skip / 0 fail**.
- Introduced **zero** new test failures; the only transient regression was the existing `store-closed-on-new-bar.test.ts` asserting `closed` mutation persistence, which exposed the proxy-link bug above and was fixed before commit.

Phase 2 — Rendering & data-plane performance (landed)

- **Bar-plane append no longer copies the whole window.** Added a targeted `setBarPlaneAppend(bar, index)` path in `src/stores/bars.ts` used by `appendBar` for the common in-place append. The previous full-array

`setBarPlaneFull` is still used for initial loads, slot restores, and rolling-window slides, plus a fallback when the plane length has drifted from `store.bars` (e.g. tests that write `store.bars` directly).

- **`setDataToChart` skips duplicate volume writes.** When the candle series paint is skipped because the bar window is unchanged (`skipSeries`), the volume series `setData` is now skipped too — it derives from the same bars.
- **Compare overlay avoids identical `setData`.** `applyCompareOverlay` (`src/chart/compare-overlay.ts`) now caches the last result from `buildCompareSeriesData` and short-circuits the LWC write when the built data is identical (which happens on unrelated `chartDataGen`/label-pref effect re-runs).
- **On-chain scalar overlays avoid identical `setData`.** `applyOnchainOverlays` (`src/chart/onchain-overlay.ts`) fingerprints each line's point array and skips `series.setData` when the fingerprint matches the last paint, while still re-applying visibility options.
- **On-chain event markers avoid full rebuilds.** `applyOnchainEventMarkers` (`src/chart/onchain-events.ts`) fingerprints the event set and skips the invisible host-series `setData`/marker-plugin rebuild when unchanged.
- **Ranging-aware overlay suppression in ChartHost.** Added a reactive `isRanging` signal in `src/chart/ChartHost.tsx` that polls `PaneManager.isRanging()` while mounted. The compare, on-chain scalar, and on-chain event effects now suppress full writes while the user is panning/zooming and flush the latest state once the gesture ends.

Validation after Phase 2

- `bun run typecheck:all` — clean.
- `bun run lint` — exits 0.
- `bun run test:unit` — **4708 pass / 10 skip / 0 fail**.

Phase 3 — Engine-boundary performance (landed)

- **Off-thread engine JSON parsing.** Added `parseEngineJsonAsync` in `src/engines/json-sanitize.ts` and a dedicated `src/engines/json-parse-worker.ts`. Payloads above 256 KB are parsed in a module Worker using the same `sanitizeNonFiniteJson` logic, so main-thread blocking drops for large `/run`, `/run/batch`, and Pyodide result strings. Wired into:
  - `src/engines/live-sessions.ts` (create/tick/append/status)
  - `src/engines/engine-ws.ts` (inbound WebSocket frames)
  - `src/engines/catalog.ts` (REST `/run` and Pyodide local path)
  - `src/engines/batch.ts` (batch `/run/batch` envelope)
- **Typed-result contract for Pyodide.** The local Pyodide path now runs its result JSON through `parseEngineJson` and lifts it with `normalizeEngineResult` (`src/indicators/run-helpers.ts`) before returning, so REST/WS/Pyodide all present the same `NormalizedRunResult` shape to the apply pipeline.
- **In-memory result cache keyed by `(scriptId, barsGen, inputs)`.** New `src/indicators/run-cache.ts` memoizes successful non-live runs using a stable fingerprint of engine id, script id/script source hash, resolved inputs, engine config, bar-plane generation, bar window length, and resolved library sources. It is bounded (16 entries, 30 s TTL), clones cached values on store and retrieve to protect apply-stage mutations, and is automatically disabled in test environments to avoid cross-test pollution. Wired into `runScript` in `src/indicators/runner.ts`.
- **Pyodide streaming.** True chunked/incremental result streaming is still outstanding; the immediate win is that the final large Pyodide JSON string now parses off the main thread. Streaming evaluation remains on the Phase 5 backlog.

Validation after Phase 3

- `bun run typecheck:all` — clean.
- `bun run lint` — exits 0.
- `bun run test:unit` — **4708 pass / 10 skip / 0 fail**.

Phase 4 — UX / visual polish (landed)

- **Motion + safe-area tokens.** `@theme` now owns `--ui-motion-duration-fast/base/slow` and `--ui-safe-top/bottom/left/right`. Raw `env(safe-area-inset-*)` call sites (mobile chrome, dialog sheets, `FloatableShell` phone geometry) read the tokens. Reduced motion collapses the duration tokens to `0.01ms` on `:root, html` and globally shortens animation/transition duration so `transitionend` still fires.
- **Drawing rail is compositor-only.** `.axis-draw-rail` fades with `opacity var(--ui-motion-duration-fast)` instead of `max-height` / `max-width`. Hover-slide layout transitions (`width` / `height` / `flex-basis` / `min-width`) go through `hoverSlideTransition()` and are skipped when `prefersReducedMotion()` is true; CSS also sets `transition: none` on `.axis-panel-hover-slide` under reduced motion. Peek/expand still happens, instantly.
- **Theme tokens on chart scale chrome.** `ChartScaleControls` uses `--color-border`, `--color-accent`, `--color-text(-dim)`, `--color-bg-panel` instead of hard-coded hex.
- **Shared `Spinner`.** `src/ui/Spinner.tsx` wraps the loader icon with `role="status"`, `aria-busy`, a stable `data-testid="axis-spinner"`, and a default "Loading" label. Wired into IndicatorCard, OnChainPanel, ResultsModal, ScriptSettingsModal, StorageChangeDialog, and SymbolModal. Busy *refresh* glyphs keep `Icons.refresh` + `animate-spin`.
- **Coarse-pointer topbar targets.** `@media (pointer: coarse)` raises command-bar / module / engine-switch buttons to a 40×40 px minimum with `touch-action: manipulation`.
- **Modal focus restore + aria-hidden siblings.** `installFocusTrap` accepts `returnElement` and hides ancestor-chain siblings with `aria-hidden="true"` while the trap is active (already-hidden nodes stay hidden; previous values including `false` are restored on dispose). Solid `ref` fires before insert, so the sibling walk retries on a microtask once the dialog is in-tree. `Modal` passes `returnElement` through. Dispose order: unhide, then focus `returnElement` or the previously focused element. Missing `document.body` is a no-op so test doubles keep working.

Validation after Phase 4

- `bun run typecheck:all` — clean (566 app + 29 worker + 39 CLI files).
- `bun run lint` — exits 0 (warnings remain; no errors). New `!important` on the global reduced-motion rule is intentional.
- Phase 4 pins: `tests/spinner.test.ts`, `tests/phase4-ux-tokens.test.ts`, plus extensions of `tests/focus-trap.test.ts`, `tests/ui-modal-shell.test.ts`, `tests/panel-hover-slide.test.ts` — **36 pass**.
- `bun run test:unit` — **4723 pass / 10 skip / 1 fail**. The failure is `data-source-manager > applyCachedToChart with onlyIfCurrent skips paint for a stale symbol` timing out at 5 s (reproduced in isolation at ~6 s). It does not import Phase 4 UI files; treat it as a pre-existing slow DSM path, not a Phase 4 regression.

Phase 5 — Continuous quality gates (landed)

- **`bun run perf:audit`.** `scripts/perf-audit.mjs` runs `typecheck`, a hot-path unit slice (`store-append-scale`, `live-append`, `store-live-frame`, `paint-latency`), then `AXIS_BENCH=1 bun test tests/bench/`. It scrapes `AXIS_BENCH {json}` lines from bench stdout and writes `reports/perf-audit.json` (gitignored). `--full` / `bun`

`run perf:audit:full` also runs `test:unit` and Playwright `@smoke`. `--skip-typecheck` / `--skip-bench` exist for iterating on one side.

- **CI firehose budget.** `.github/workflows/ci.yml` adds a parallel **Firehose budget** job: `bun run test:bench` with `AXIS_BENCH=1`, 10-minute job timeout. The existing `expect(sample.ms).toBeLessThan(FIREHOSE_SOFT_BUDGET_MS)` (120 s) fails the job if the 500k-tick append path regresses into minutes. Dataset linear-merge keeps its 5 s soft budget.
- **ADR-017 — Narrow bar plane.** `docs/architecture/adrs.mdx` records `barPlane` (`bars` / `tip` / `length` / `gen`), the clone-on-full-write rule, and that `store.bars` stays the engine/eval source during migration.
- **ADR-018 — Engine result parse + typed contract.** Records the 256 KB Worker parse threshold, `normalizeEngineResult` as the apply envelope, and the bounded run-cache. Chunked Pyodide evaluation stays future work.

Validation after Phase 5

- `bun run typecheck:all` — clean (566 app + 29 worker + 39 CLI files).
- Phase 5 pins: `tests/perf-audit.test.ts` — **9 pass**.
- `bun run perf:audit --skip-typecheck` (this host, 2026-11-01): hot-path unit 441 ms; **appendBar firehose 500k ticks in 19.1 s** (26.1k ops/s, budget 120 s); dataset linear merge 50 pages in 829 ms (budget 5 s). Report: `reports/perf-audit.json`.

9. Next move

The six-phase audit plan is implemented through quality gates. Follow-ups that remain open: chunked Pyodide result streaming, typed-array engine envelopes, a nightly `axis-nightly.yml` if full e2e should stay off PR CI, and shrinking the DSM `applyCachedToChart` timeout. Keep requiring before/after measurements for any new hot-path change.

TOPOLOGIES

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Topologies" description: "AXIS deploy and dev topologies: Vite+Flask, static PWA, Cloudflare® Pages+Worker, offline Pyodide lab, and local Bun sidecar."

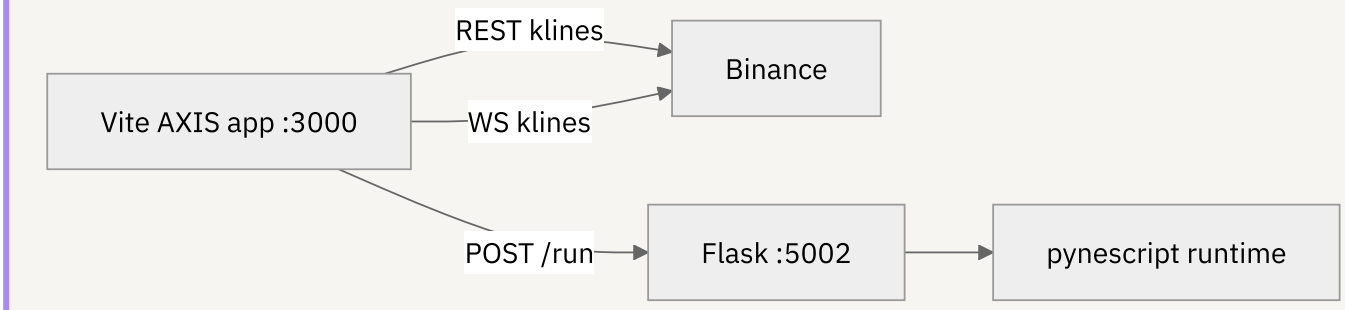
TOPOLOGIES

Concrete process and network shapes for AXIS. Choose a topology; then compose plugins inside it (recipes).

Abstract

Topology	AXIS	Calc	Live	Library
Dev desk	Vite `:3000`	Flask `:5002`	Binance WS direct	local
Static demo	`axis_pwa_server` `:8081`	Flask or remote	venue / mock	local
Edge	CF Pages	Worker → Flask	DO fan-out optional	cloud
Offline lab	any AXIS	Pyodide	mock-poll	local
Dev desk + sidecar	Vite `:3000`	Flask `:5002`	Bun sidecar `:5003`	local

Topology A — Dev desk

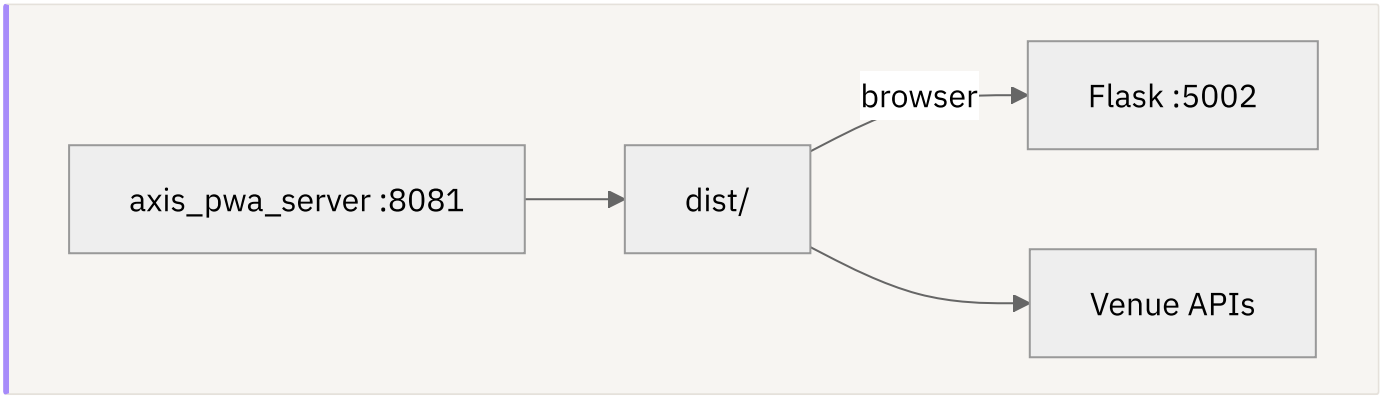


Bring-up

```
make run
bun install && bun run dev
```

Notes: Vite may proxy `/run`; Settings endpoint can still point absolute at `:5002`. CORS must allow Vite origin.

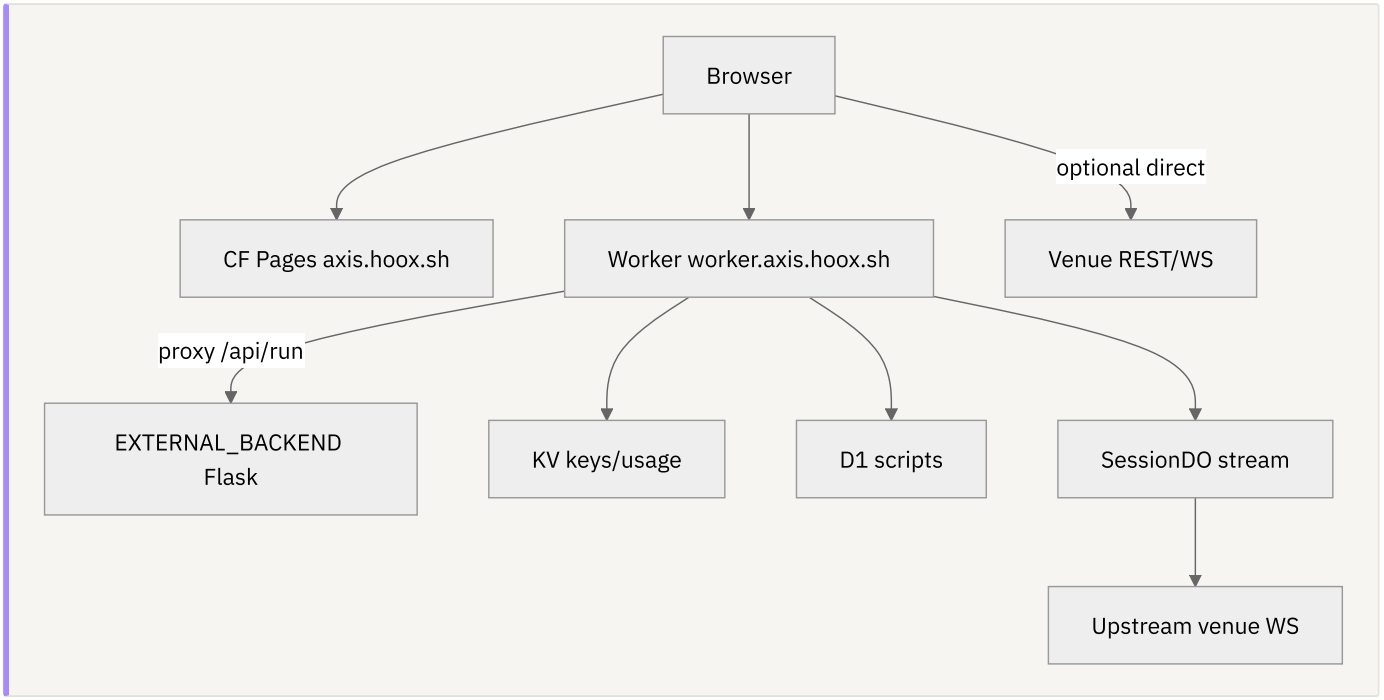
Topology B — Static PWA + Pro API




```
bun run build
python3 axis_pwa_server.py
make run   # separate process
```

VPS demo pattern: systemd units for `axis-pwa` and `pynescript-api`; `ALLOWED_ORIGINS` includes AXIS origin.

Topology C — Cloudflare® AXIS at the edge



Deploy sketch

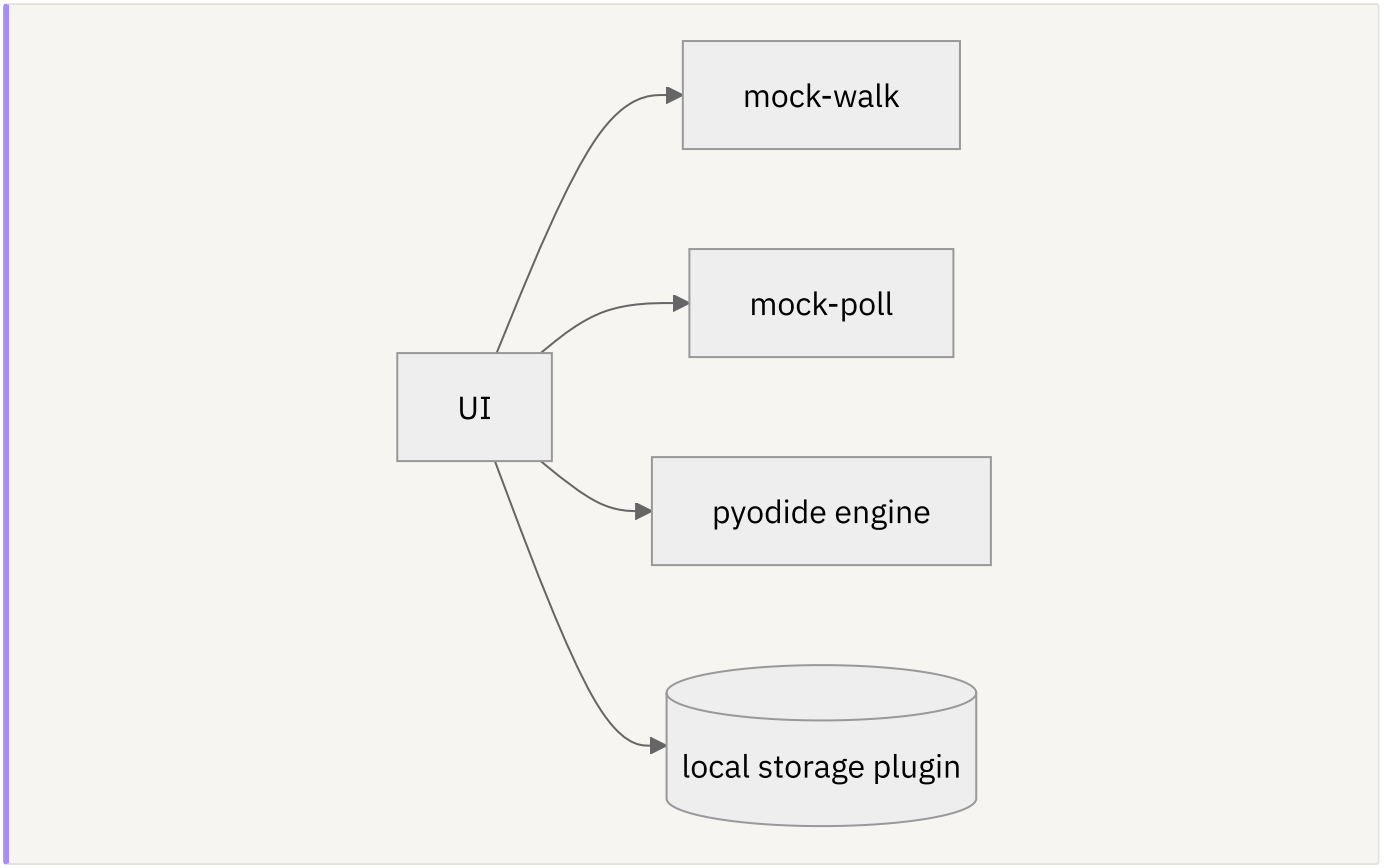
```
axis deploy
# or: cd worker && bun run deploy
bun run build
wrangler pages deploy dist --project-name=axis
```

Ids: Pages project `axis` at `https://axis.hoox.sh`; Worker script `worker-axis` at `https://worker.axis.hoox.sh` (ADR-008).

AXIS config:

- Engine `server`, endpoint = Worker origin
- Storage `cloud`, same origin + API key
- Stream: direct venue or DO-backed plugin

Topology D — Offline lab



No Flask, no venue. Requires vendored pyodide + wheels on origin once.

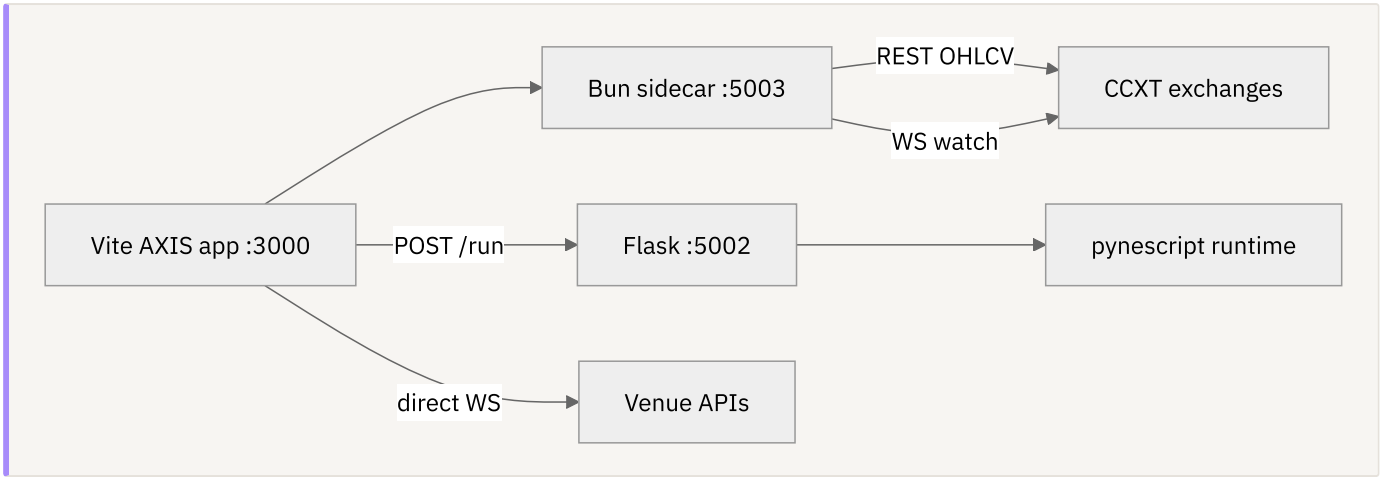
Topology E — Git-centric research

Any of A–C for calc/data; storage axis = `git`. Forges are orthogonal network peers:

```
AXIS --storage:git--> api.github.com | gitlab
AXIS --engine:server--> Flask/Worker
AXIS --source--> venue or CSV
```

Topology F — Dev desk + Bun sidecar (CCXT Pro)

For users who want authenticated WS feeds from any CCXT-supported exchange without running Flask. The sidecar (`packages/datafeed/`) mirrors the PYNE datafeed gateway contract.




```
bun run dev           # Vite :3000
bun run dev:datafeed  # Bun sidecar :5003
# in separate terminals, or combine in a Procfile
```

Gateway contract (same as PYNE datafeed):

Endpoint	Method	Description
`/datafeed/ohlcv?exchange=&symbol=&timeframe=`	GET	Historical OHLCV
`/datafeed/markets?exchange=`	GET	Available markets
`/datafeed/watch?exchange=&symbol=&timeframe=`	WS	Live bar stream
`/datafeed/session`	POST	Store credentials (RAM only)
`/health`	GET	Sidecar health

Notes: Credentials stay in RAM only (never persisted). The sidecar is optional — public venues work without it via direct browser WS.

Port map (defaults)

Service	Port
Vite dev	3000
Flask Pro API	5002
Bun datafeed sidecar	5003
Static PWA server	8081
Wrangler Worker	8787

Comparison

Concern	A Dev	B Static	C Edge	D Offline	F Sidecar
HMR	yes	no	no	n/a	yes
PWA SW realism	partial	full	full	full	partial
Multi-tenant keys	no	no	yes	no	local only
Ops complexity	low	medium	high	low	low
Fidelity	high (Flask)	high	high via proxy	engine-dependent	high (CCXT Pro)

Failure modes by topology

Topology	Typical break
A	Flask not running; CORS localhost vs 127.0.0.1 mismatch
B	Stale SW; missing `dist` pyodide assets
C	Unbound KV/D1; wrong project name; EXTERNAL_BACKEND down
D	First visit offline; SPA fallback for wheels

Internals

Path	Role
`vite.config.ts`	Dev/build
`axis_pwa_server.py`	Static host
`worker/wrangler.toml`	CF name + bindings
`packages/cli/`	AXIS CLI (setup / deploy / health)
`worker/README.md`	Edge ops

See also

- ADR-008, 009, 010, 012
- Installation
- Worker
- DevOps

STATE NAMESPACES

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "State namespaces" description: "AXIS persistence keys, key migration, pluginsConfig, editor docs, library IDB, and URL hash state."

STATE NAMESPACES

AXIS persistence is multi-homed: **layout/config** in localStorage, **script library** in IndexedDB (or remotes), **ephemeral run data** in memory, optional **URL hash** for shareable slices.

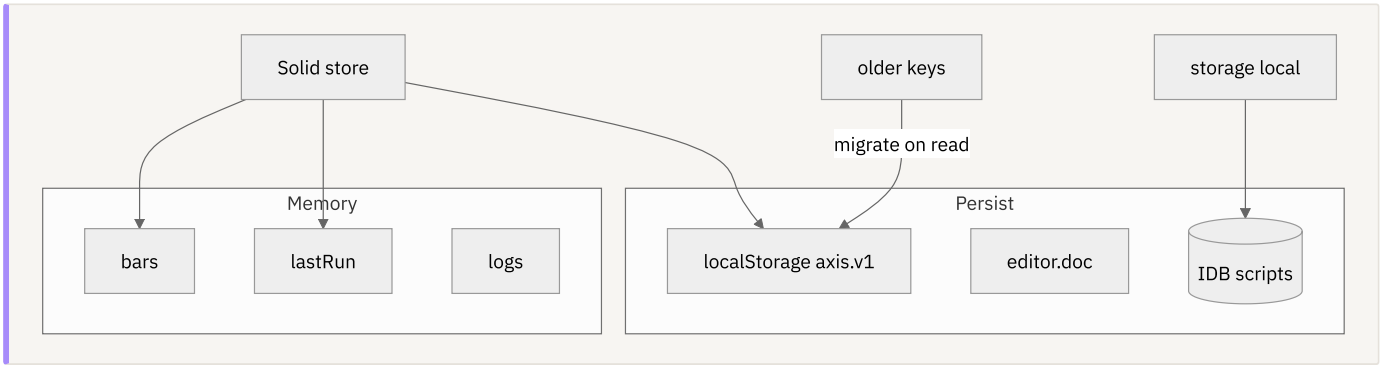
Abstract

Namespace	Medium	Purpose
`pynescript.axis.v1`	localStorage	App shell state
`pynescript.axis.editor.doc`	localStorage	Editor document backup
`pynescript.axis.storage`	IndexedDB	Local library
`pynescript.axis.library.v1`	localStorage fallback	Library if no IDB
`pynescript.axis.library.draft`	localStorage	Draft buffer
Plugin install list	localStorage	URL plugins restore
`store.bars` / `lastRun` / `logs`	memory	Session only

Contract / product identifiers (not storage keys):

```
pynescript.axis.plugins.v1   # conceptual plugin contract id
axis                         # CF Pages project id
axis.hoox.sh                 # Pages custom domain
worker-axis                  # CF Worker script id
worker.axis.hoox.sh          # Worker custom domain
```

Conceptual model



App state key `pynescript.axis.v1`

Solid store (`src/store/index.ts`):

Persisted (representative):

- Market: `symbol` , `interval` , `exchange` , `source` , `engine` , `endpoint`
- `activePlugins` `{ source, stream, engine, storage }`
- `pluginsConfig`
- Layout: `theme` , `editor` , `watchlist` (named lists + active symbols), panel open/width/height
- `panes` , `scripts` (indicator list metadata)
- `live.streamId` (and related live flags carefully)
- `drawings` (user annotations)
- Flat mirrors: `source` / `engine` aligned via `setActivePlugin`

Explicitly omitted on persist:

- `bars`
- `lastRun`
- `logs`

Debounced write (~200ms).

Legacy migration

Read order:

1. `pynescript.axis.v1`
2. else `pynescript.axis.v2`

On older-key hit: copy forward into `pynescript.axis.v1`.

The pre-Solid vanilla `state.js` shell (removed in 2.21.0) used the same STORAGE_KEY constant. It is gone; only the Solid store path remains.

pluginsConfig

Map of configuration objects. Preferred keys:

```
`${kind}:${id}`   e.g. storage:cloud, storage:git, engine:server
```

Bare ids may still be read for compatibility (`cloud`, `git`). Values feed `configSchema` resolution inside plugins.

Sensitive fields: `apiKey`, git `token` — browser-local; treat profile as secret store.

Editor document

Key	Role
`pynescript.axis.editor.doc`	Draft / shared doc
Bridge messages	Popout synchronization

Popout mode uses `editor-bridge` + shared storage so detached windows share source without re-fetching library.

Local library IDB

Constant	Value
DB name	`pynescript.axis.storage`
Version	1
Stores	`scripts`, `kv`

Migration flags (`pynescript.axis.library.migrated`) prevent re-import loops from older library keys:

- `pynescript.axis.library.v1`
- `pynescript.axis.library.legacy`

Cloud / git (remote namespaces)

Not browser keys—server-side partitions:

Backend	Partitioning
cloud	Hash of API key on Worker; D1 table or memory
git	`owner/repo@branch` + `basePath`

Revisions: cloud may expose `revision` / If-Match; git uses blob SHAs via forge APIs.

URL hash state (legacy helper)

`src/state-hash.js` (legacy state path):

Feature	Detail
Keys	symbol, interval, engine, source, stream, timeRange
Script	base64 in hash if short
Cap	~2000 chars
API	`applyHashState`, `pushHashState`, `watchHashState`

Solid-first product may not wire this by default; treat as optional share mechanism. Prefer library export + recipe docs for durable shares.

Invariants

1. Migration is **read-repair** to AXIS keys.
2. Never persist full OHLCV in `axis.v1`.
3. `setActivePlugin` keeps flat fields coherent.
4. Logs panel open state forced closed on hydrate (noise control).
5. Drawing tool resets to `cursor` on hydrate; geometries restore.

Failure modes

Issue	Effect	Mitigation
QuotaExceeded	persist no-op	Export library; shrink drawings
Poisoned JSON	load defaults	Security tests; clear key
Split profiles	“missing” library	Same browser profile
Stale activePlugins id	missing plugin	Fall back / reselect built-in

Internals

Path	Role
`src/store/index.ts`	Solid persistence
`src/store/types.ts`	`AppState`
`src/state.js`	Legacy state
`src/state-hash.js`	Hash sync
`src/storage/local.ts`	IDB library
`src/plugins/loader.ts`	Installed plugin list

See also

- ADR-011
- AXIS store
- Script library

DRAWINGS AND PLOT PARITY

Plan to make user drawing tools and Pine plot/drawing outputs match PYNE and render with correct pane, color, geometry, and settings.

DRAWINGS AND PLOT PARITY

AXIS has two independent render paths:

Path	Source	Paint
User drawings	Toolbar tools	`DrawingLayer` user SVG group, editable
Pine output	PYNE `RunResult`	plots → LWC series; shapes/barcolor → candle markers; bgcolor → histogram; fill → SVG; line/label/box/polyline/linefill → script SVG; tables → chart HUD or bottom Tables panel

This plan is the work list from a four-agent audit of AXIS (`src/chart` , `src/results` , `src/indicators/runner.ts`) against PYNE (`pynescript` `drawing.py` , `evaluator.py` , `host.py`).

Phases

P0 — AXIS correctness (this pass)

User tools and apply-pipeline bugs that do not wait on a PYNE export change.

1. **1-point body-move** — `shiftDrawing` only special-cased `text` / `priceLabel` . Shift any drawing with `p1` (and `points[]` when present): note, flag, crossline, anchoredText, arrowMarkUp/Down.
2. **p3 / extra vertex handles** — `hitTestHandle` / `resizeDrawing` only p1/p2. Add `p3` (and remaining `points[i]` as p3 for the third anchor). Paint extra handles after registry paint.
3. **Linefill GC** — `garbageCollectScriptDrawings` early-return ignored `skip.linefill` .
4. **Polyline / linefill overlay** — copy `force_overlay` and polyline `fill_color` → `bgcolor` .
5. **Stale SVG** — silent live re-run with `drawings: []` must clear script drawings.
6. **Pane routing** — `bgcolor` and `fill()` use the script pane when `overlay=false` (`syncBgcolorBands(paneId)` , `ensureScriptPaneLayer` for fills). `plotshape` / `plotchar` / `plotarrow` attach to a series on that pane when possible; otherwise stay on price (LWC markers need a host series).

P1 — User-tool settings (AXIS)

Done:

- Shared style bar: color (presets + custom), width, dash, fill for shapes / ranges / fib / highlighter / positions.
- Per-tool extras via `src/chart/drawings/tool-settings.ts` + settings popover (`axis-drawing-settings`).
- Last-used style per kind (`drawingPrefs.byKind`).
- `extendLeft` / `extendRight` UI + paint (trend, ray, extend, info line, channel, fib).
- Fib level editor, reverse, show % / price, fill between levels.
- Text re-edit (popover + double-click), font size.
- Per-drawing lock in the style bar (unlock bypasses `updateSelected` lock gate).
- Long/short risk-reward (`meta.rr`).

- Highlighter width honors `style.width` (create default 8).
- hray: 1-click (price only).
- xabcd / headShoulders: commit on 5th point.

P2 — PYNE export (pynescript repo)

AXIS cannot paint fields the engine never serializes.

Gap	Where	Needed on the wire
Table cell keys `(row,col)` vs `(col,row)`	`drawing.py` `table.cell` vs `cell_set_*`	One index order
Table style	`export_for_api`	halign, valign, width, tooltip, text_size, borders (AXIS already parses `width` / `height` / `tooltip` / `merged_cells` / `force_overlay` when present; bottom panel uses them for proportional columns, hover tips, and merges)
`table.cell` kwargs	ctor	width, halign, valign, size
plotshape Y / `location.absolute`	`_coerce_plot_shape`	keep numeric series (or `{y, on}` cells)
plotbar / plotcandle	evaluator	open/high/low/close (or sibling titles in `plot_meta`)
`fill()` compile / untitled plots	host + evaluator	`plot1` / `plot2` series keys + per-bar fill color column
Positional `plot(..., linewidth)`	`evaluator.py` ~425	Pine order: series, title, color, linewidth, style
plotarrow	evaluator	`colorup` / `colordown` / minheight / maxheight
`offset`, `force_overlay`, `histbase`	`_pack_interpret_plot_columns`	remaining plot_meta fields (display is packed)
Polyline	export	`fill_color`, `curved`, `force_overlay`
Box / label extras	export	closed, extend, text_*, tooltip, border

P3 — Remaining Pine paint

Done this pass:

- Hide-drawings toggle includes script SVG + fill bands (all pane layers).
- Label `yloc` pad (8px off wick); `label.style_text_outline` as outlined text.
- First `addIndicator` re-owns plot fills / script drawings / plotshape (`EDITOR_RUN_KEY` → script id).
- Histogram `histbase` from `plot_meta` (default 0). PYNE still needs to serialize it for live values.
- Pine `display.*` (`none` / `pane` / `data_window` / `price_scale` / `status_line` / `all`) on plot, hline, shapes, fill, bgcolor, barcolor.

Still open:

- `location.top` / `bottom` = pane edges, not bar (LWC markers only have above/below/in bar).
- Per-bar `plot(..., color=expr)`, plotarrow length (`colorup` / `colordown` / `minheight` / `maxheight` wait on PYNE).

Pine tables: bottom Tables panel (shipped)

`table.*` output paints in the chart HUD (`PyneTableHud`) by default. Per script, **Move tables to bottom** (chart ⋯ menu, pane badge ⋯, command palette) relocates that script's tables to `ui/TablesPanel.tsx`:

- One tab per script, stacked vertically in creation order regardless of `position` / `force_overlay`.
- Proportional column widths from `table.cell(width=…)` (`bottomPanelColumnFractions` in `chart/pine-tables.ts`), tooltips + merged cells preserved, text selectable.
- Placement persists per script in `store.pineTablesLocation`; the HUD hides moved tables. Move back via **Move tables to chart**.

Invariants

- User drawings ≠ Pine drawings. Do not share hit-testing or magnet with script SVG (`pointer-events: none`).
- `overlay=false` plots, fills, bgcolor, and shapes belong on `ind_*` unless `force_overlay`.
- `barcolor()` stays on price candles.
- Color sanitize: `#rgb(a)`, `#rrggbb(aa)`, `rgb/rgba`. Unknown → VOID indigo for **user** tools; Pine should keep engine hex (including `#RRGGBBAA`).
- GC caps match Pine: 50 default, 500 lines/labels/boxes, 100 polylines. Linefill cap is AXIS-only until PYNE emits `max_lines_fills_count`.

Tests

- `shiftDrawing` / `resizeDrawing` for 1-pt and 3-pt kinds.
- Linefill GC overflow.
- Polyline `force_overlay` + `fill_color`.
- Runner: bgcolor/fill target pane when `overlay=false` (unit with a fake manager if needed).
- PYNE: table cell key, fill meta, plotshape Y (in the pynescript repo).

HP ENGINE

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Live chart engine" description: "Lightweight Charts paints the chart. One ingest worker owns /ws/live and decodes AXB1. The shared-memory ring and GPU overlay were removed."

LIVE CHART ENGINE

Lightweight Charts paints candles, indicators, and fills. There is no second candle painter.

One browser worker owns the engine live socket. It decodes JSON and AXB1 off the window thread and posts frames to the chart. That path is the default: `axis.engine.transport` unset or `worker`. Set the flag to `json` and reload to keep the main-thread client. If the worker cannot start, the chart uses that client once and counts the fallback.

What stayed

Piece	Where	Role
AXB1 codec	`src/hp/axb1/`	Binary frames on the worker socket. Prices stay int64 ticks.
Ingest worker	`src/hp/workers/`	Owns `/ws/live`, coalesces forming tips, posts decoded frames.
Execution context	`src/hp/calc/execution-context.ts`	A forming tip does not commit a closed bar.
Isolation headers	`src/hp/isolation-headers.ts`	COOP / COEP / CORP for the static hosts and the offline shell.

What was removed

The shared-memory hot ring, the cold page slot, the WebGL2 and WebGPU candle overlay, the calc worker, the HUD chip, and the `axis.engine.hp.*` flags are gone. Settings has no high-performance section. `hp.get`, `hp.sample`, and `dev.trace` are not MCP capabilities.

Stored `axis.engine.hp.enabled`, `axis.engine.hp.gpu`, and `axis.engine.hp.sab` values are ignored.